

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DeFi Strategy	Documentation quality	Medium
Timeline	2026-08-25 through 2026-08-31	Test quality	High
Language	Solidity	Total Findings	12 Fixed: 8 Acknowledged: 3 Mitigated: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0
Source Code	bloqpriv/vesper-strategies ⓘ #977e218 ⓘ	Medium severity findings ⓘ	3 Fixed: 3
Auditors	- Zeeshan Meghji Senior Auditing Engineer - Alejandro Padilla Senior Auditing Engineer	Low severity findings ⓘ	5 Fixed: 3 Acknowledged: 1 Mitigated: 1
		Undetermined severity findings ⓘ	0
		Informational findings ⓘ	4 Fixed: 2 Acknowledged: 2

Summary of Findings

Quantstamp reviewed the Vetro Concrete strategy contracts, which manage capital allocated from Vetro pools into external Concrete vaults. The strategies support synchronous and asynchronous vault integrations, including depositing idle collateral, reporting profit and loss back to the pool, servicing withdrawals, and, for asynchronous vaults, tracking and processing queued redemptions across Concrete epochs. A keeper-managed variant also allows principal movement to be handled explicitly through privileged keeper operations.

The most significant findings relate to strategy accounting and the interaction between the strategies and their external dependencies. **VET-1** identifies a mismatch between the strategy's profit-reporting logic and the consuming pool's requirements that can cause rebalancing to revert under expected operating conditions. **VET-2** describes how a redemption moved between Concrete epochs can temporarily disappear from strategy accounting, causing a false loss to be reported and reducing the strategy's allocation. Other findings address loss-cap configuration, deposit-cap enforcement, withdrawal sizing, unbounded epoch tracking, and post-deployment configuration. We have also included several recommendations in the Auditor Suggestions section to reduce operational complexity and make integration assumptions more explicit.

Several important recommendations are included in the Operational Considerations section of the report. The strategies rely on external Concrete vault behavior, pool governance, keeper operations, and, in some cases, manual epoch processing, all of which can materially affect liquidity, accounting, and strategy availability. We therefore recommend careful monitoring of vault configuration and queue state, explicit operational procedures for keeper-managed actions and epoch tracking, and strong governance controls around strategy upgrades and other sensitive operations. The scope of this audit was limited to the on-chain Vetro strategy contracts; the consuming pool contracts and external Concrete vault implementations were outside the formal audit scope.

The test suite is thorough and includes both unit and integration tests across the different Concrete strategy variants and deployed vault configurations, with relatively high branch coverage of 91.5%. The review nevertheless identified an important limitation in the pool mock used by the tests, which prevented the existing suite from detecting **VET-1** despite otherwise broad coverage.

Fix-Review Update 2026-09-22:

Most issues have been fixed by the development team. **VET-7** has been sufficiently mitigated. **VET-8**, **VET-9**, and **VET-12** were acknowledged as either out of scope or as part of the intended design.

ID	DESCRIPTION	SEVERITY	STATUS
VET-1	<code>_rebalance()</code> Sizes Reportable Profit Against The Credit Line, Which <code>reportEarning()</code> Does Not Permit	• Medium ⓘ	Fixed
VET-2	Epoch Migration Can Hide Pending Redemptions, Causing the Strategy to Report a False Loss and Reduce Its Allocation	• Medium ⓘ	Fixed
VET-3	Enabling the Redeem Queue Can Turn Synchronous Withdrawals Into Untracked Requests, Causing False Losses	• Medium ⓘ	Fixed
VET-4	A Zero Default Loss Cap Causes Any Shortfall to Block Rebalancing	• Low ⓘ	Fixed
VET-5	A Failed Claim Causes the Strategy to Withdraw Too Little and Remain Above Its Deposit Cap	• Low ⓘ	Fixed
VET-6	<code>_tryWithdraw()</code> Does Not Apply The Vault's Minimum-Withdraw Floor To The Requested Amount	• Low ⓘ	Fixed
VET-7	Unbounded <code>_pendingEpochs</code> Set Is Iterated	• Low ⓘ	Mitigated
VET-8	Managed-vault deposit caps do not bound ongoing vault exposure	• Low ⓘ	Acknowledged
VET-9	Missing Post-Deployment Configuration Can Leave Strategies Silently Non-Operational	• Informational ⓘ	Acknowledged
VET-10	Missing Events Make Principal Movements and Skipped Withdrawals Harder to Monitor	• Informational ⓘ	Fixed
VET-11	<code>_syncWithdrawable()</code> <code>maxWithdraw</code> Comment Overstates The Pre-v1.5.0 Over-Report	• Informational ⓘ	Fixed
VET-12	<code>_approveToken()</code> Couples The Pool And End-Protocol Allowances To A Single Parameter	• Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The audit covers the contracts in the `contracts/strategies/concrete/*` folder only. The audit does not include the Vetro pool contracts or the Concrete vault contracts.

Files Included

Repo: `https://github.com/bloqpriv/vesper-strategies`

Included Paths: `contracts/strategies/concrete`

Operational Considerations

`ConcreteVault` Must Not Be Connected to a Queue-Capable Concrete Vault

`ConcreteVault` is designed exclusively for Concrete vaults whose withdrawals always settle synchronously, such as vaults using `ConcreteStandardVaultImpl`. It does not track redemption epochs or account for liabilities created by a redeem queue. A vault using `ConcreteAsyncVaultImpl` should therefore be integrated through `ConcreteAsyncVault`, even if its queue is currently inactive.

If the queue is subsequently enabled, a withdrawal can escrow the strategy's shares and create a redemption request without immediately transferring assets. `ConcreteVault` treats the call as successful but does not track the request, causing its value to disappear from TVL and potentially be reported as a false loss. In addition, legacy queue-capable implementations may include assets reserved for processed redemptions in `maxWithdraw()`, causing `ConcreteVault` to overstate synchronous liquidity and request withdrawals that cannot settle.

Deployment procedures should verify the Concrete implementation type and reject `ConcreteVault` integrations with any queue-capable implementation; checking only the queue's current status is insufficient. Concrete upgrades and implementation changes should also be monitored. If queue capability is introduced for a live integration, the strategy should be migrated to `ConcreteAsyncVault` before further withdrawals are attempted. Any requests created before migration must then be cancelled or tracked and processed, and any resulting false loss or debt-ratio change must be reconciled separately.

Capital in an Async Concrete Strategy Cannot Be Withdrawn on Demand

Capital in an Async Concrete Strategy Cannot Be Withdrawn on Demand. `ConcreteAsyncVault` deposits into a Concrete vault whose redeem queue, when active, processes withdrawals through epochs that must be closed and processed by Concrete's operator. While the queue is active, the strategy has no synchronous exit from its Concrete position. If a pool withdrawal exceeds the strategy's idle balance, no additional assets can be recovered immediately and the user's withdrawal reverts.

The strategy will request the required redemption during a subsequent rebalance, but the time until those assets become available is unbounded. Closing and processing an epoch requires a manual, role-gated Concrete operation with no on-chain deadline, and the strategy cannot accelerate this process.

This delay is also not known on-chain in advance: the strategy does not expose how long unavailable capital will remain locked in the redemption process. Failed withdrawals only provide a signal when a transaction is actually submitted, so monitoring should track available liquidity directly rather than relying solely on withdrawal failures.

Operators should account for this liquidity constraint when sizing allocations to async Concrete vaults, particularly during periods of high withdrawal demand or market stress.

Queued Redeems on Newer Concrete Vaults are Discretionary

The newer vault implementation lets a privileged Concrete operator settle a redeem request sitting in the currently open epoch, paying out at a discount to its value and burning the corresponding shares. That discount is capped — currently at 5% — but the cap is set by the vault operator and can be raised as high as 100%, taking effect immediately on vaults where timelock enforcement is not yet active; on `ctDefiUSDT` the roles that raise the cap and execute the settlement are held by the same key as vault ownership. There is no on-chain defence, since the vault burns shares it already holds in escrow and the strategy receives neither a callback nor a veto. Exposure is bounded by the shares actually queued rather than by the whole position, and `_afterReport()` requests only what payback or the deposit cap requires, so the risk concentrates on deliberate unwinds: exit across several epochs rather than a single `requestRedeem()` of the full balance. Monitor the vault for settlement events and cap changes; `cancelRequest(epochID)` is the response lever while the epoch is still open, subject to the operator not having

disabled cancellation. A discount surfaces as a loss at the next rebalance, so keep `maxLossBps` tight and treat the resulting `LossExceedsCap` revert as the intended alarm, clearing it deliberately with `reportLoss()` rather than by widening the cap.

Deposit Whitelist Hooks are Invisible to `maxDeposit()`

Some Concrete vaults configure a deposit hook — typically a depositor whitelist — that gates `deposit()` and `mint()` independently of the vault's configured deposit limits. No vault implementation's `maxDeposit()` consults hooks; it reflects only those limits, so the gate is invisible to any integrator sizing a deposit from it. `_toSizedDepositAssets()` therefore sizes a full deposit, the vault reverts inside the hook, and `_tryDeposit()`'s catch swallows it — `rebalance()` succeeds, reports no loss, and quietly leaves the collateral idle. Any strategy deploying against a hooked vault must be enrolled on the hook contract by Concrete before funding. Because the vault operator can also add or swap hooks after deployment, a vault that accepts deposits today can start refusing them later, so treat `VaultActionSkipped(IERC4626.deposit.selector, ...)` as a monitored alert rather than a one-time pre-deploy check — the `reason` bytes carry the hook's revert data and identify the cause directly.

Deposit-Lock Hooks Make `instantLiquidity()` Over-Report, and the Strategy has No Way to Shorten the Lock

Some Concrete vaults register a hook that time-locks freshly minted shares. `maxWithdraw()` does not consult hooks, so `_syncWithdrawable()` returns the locked position in full, `instantLiquidity()` advertises it, and the vault's refusal is swallowed into `VaultActionSkipped` — the pool simply receives whatever idle collateral the strategy holds. `ConcreteVault`'s `NatSpec` acknowledges this, but describes it as an edge case; in practice it is the steady state, since after funding the entire position is locked for the full duration and `_afterReport()` redeposits idle on every rebalance, rolling the locked tranche forward rather than clearing it once. Consider deploying `ConcreteAsyncManagedVault` against lock-hooked vaults, as its idle-only `instantLiquidity()` never promises vault holdings and its keeper-driven posture suits a position under a rolling lock.

The Deposit Lock also Caps how Often the Strategy Can Deposit

The same hook that time-locks fresh shares limits how many unexpired locks a single depositor may hold — currently ten — and each deposit creates one, clearing only when its lock period elapses. The strategy is therefore limited to ten deposits within any rolling seven-day window, roughly one every seventeen hours. Past that the vault rejects the deposit, `_tryDeposit()`'s catch swallows the rejection, and `rebalance()` reports success while the collateral stays idle. Nothing in the strategy is aware the limit exists, and because `_afterReport()` deposits idle on every rebalance, a keeper cadence faster than that threshold silently stops deploying capital rather than failing loudly. Both the lock count and the lock duration are set by the vault operator and can be changed after deployment, so the safe cadence is not fixed: bound the rebalance schedule against the current values with margin, and treat `VaultActionSkipped(IERC4626.deposit.selector, ...)` as the alert. It is the only on-chain signal, and its `reason` bytes distinguish a lock-count rejection from a whitelist or deposit-limit rejection.

The Keeper-Managed Variant Requires Manual Position Management

`ConcreteAsyncManagedVault` overrides `_rebalance` to report profit and loss without performing the housekeeping executed by its parent strategies. It does not automatically claim processed epochs, reduce a position above its deposit cap, or redeploy idle collateral. Instead, principal is managed manually through keeper calls to `depositToVault` and `withdrawFromVault`.

This changes the behavior of configuration such as `setDepositCap`. In the other strategies, lowering the cap causes the position to be reduced during a later rebalance. In `ConcreteAsyncManagedVault`, lowering the cap does not reduce assets already deposited. A keeper must perform the unwind separately.

Deposits and withdrawals are also asymmetric. A deposit can settle in one transaction, while an unwind may require a redemption request followed by Concrete closing and processing the relevant epoch. An accidental over-allocation therefore may not be correctable within a predictable period.

Keeper procedures should explicitly cover claiming processed epochs, enforcing the deposit cap, and managing idle collateral. Consider also calling `_beforeReport()` at the beginning of the `_rebalance` override so already-processed epochs are claimed automatically without otherwise changing principal management.

Keeper Moves in `ConcreteAsyncManagedVault` Succeed Whether or Not Anything Happened.

`depositToVault()` and `withdrawFromVault()` are the only paths by which principal moves in this variant, and neither reverts. `depositToVault()` sizes through `_toSizedDepositAssets()`, which returns `0` whenever the vault's limits leave no headroom and bails *before* the try/catch, so no event is emitted at all. A call made to `withdrawFromVault()` with withdrawals yet processed by Concrete also moves nothing, silently.

TVL Can be Silently Dropped from Moved Withdrawal Requests

Concrete can move a strategy's withdrawal request into another epoch without notifying the strategy. `ConcreteAsyncVault` continues tracking the original epoch and therefore temporarily omits the moved request from its valuation until an operator explicitly tracks the destination epoch. Whenever a request belonging to the strategy is moved, the destination epoch must be passed to `trackEpoch()` before any report, rebalance, deposit, or withdrawal that relies on strategy valuation.

Processed Epochs are Claimed All-or-Nothing, and a Failed Claim is Silent

`ConcreteAsyncVault` batches every tracked processed epoch into a single `claimWithdrawal()` call, so if that call fails nothing is claimed — not even the epochs that would have settled on their own. The failure is swallowed into `VaultActionSkipped()`, so the withdrawal does not revert; the pool simply receives whatever idle collateral the strategy already holds. The two realistic causes are a paused vault and a vault unable to cover what it owes across all processed epochs. The first is handled, since `instantLiquidity()` falls back to idle while paused. The second is not: `instantLiquidity()` counts the strategy's claimable assets without checking the vault can actually pay them. Monitor `VaultActionSkipped(IConcreteAsyncVault.claimWithdrawal.selector, ...)` alongside that headroom.

A Lapsed Off-Chain Accounting Report Stops Rebalancing Entirely

One of the vaults in the set values its position from an off-chain report with a validity window, and every valuation read reverts once that window expires. Because `tv1()` and `_rebalance()` both price the position through the vault, both revert for the duration — no profit, loss, payback,

epoch claiming or redeem queueing happens. `instantLiquidity()` and the pool's withdraw path are deliberately guarded and keep working, so user withdrawals are unaffected. A keeper seeing repeated rebalance failures against that vault should check whether the report has simply gone stale before treating it as a strategy fault; it clears when Concrete refreshes. Any off-chain monitoring that calls `tv1()` needs to tolerate the revert.

Concrete Earn V2 Is an External, Upgradeable Dependency

The strategies depend on Concrete vaults that are outside the audit scope and implemented as upgradeable proxies. At the time of review, the five target vaults used three different implementation builds, and some strategy behavior depends on the Concrete version, including the treatment of assets reserved for redemptions.

Concrete operator actions also directly affect strategy behavior. These include enabling the redeem queue, moving requests between epochs, and manually closing and processing epochs.

Concrete vaults may additionally use hooks that restrict withdrawals beyond what `maxWithdraw` reports. The strategy code documents this behavior for `ctFrontierUSDC`, which had such hooks enabled during the review.

Concrete upgrades, hook changes, and redeem-queue changes should therefore trigger a review of the affected strategy configuration and assumptions. New Concrete implementations should be evaluated before strategies are connected to them.

Timelock enforcement is active on only one vault in the set; on the rest, every privileged setter — deposit and withdrawal limits, hooks, the unwind cost cap, strategy membership — takes effect in a single transaction.

A Residual Below the Vault's Withdrawal Floor Cannot Be Recovered

Concrete enforces a per-transaction minimum withdrawal. Once a position falls below it, neither a synchronous withdrawal nor a queued redeem can be sized large enough to clear, and `tv1()` never reaches zero. Retiring or migrating a strategy ends with the governor writing the tail off via `reportLoss()`. Plan for that step in any wind-down, and be aware the floor is operator-adjustable, so the stranded amount is not fixed.

The Pool Governor Controls Critical Strategy Operations

The strategies do not maintain an independent governor and instead use `pool().governor()`. The pool governor therefore controls the strategies attached to that pool, including strategy upgrades, `reportLoss`, `setMaxLossBps`, deposit-cap changes, and keeper management.

These are significant permissions. The strategies are UUPS proxies, so the governor can replace their implementation, while `reportLoss` and `setMaxLossBps` can materially affect strategy accounting and loss handling.

The contracts do not enforce a timelock or quorum for these operations. The pool governor should therefore be protected by appropriate operational controls, such as a multisig and timelock. Strategy upgrades and `reportLoss` should be treated as production-critical operations.

The Vesper/Vetro Pool Contracts Are Outside the Audit Scope

The consuming `YieldVault` contracts are outside the audited repository and their source code was not included in the official audit hand-off.

Pool-facing behavior used in this review, including `reportEarning`, loss handling, debt-ratio changes, and withdrawal processing, was verified against the source code published for the deployed pools together with the repository's `IVesperPool` interface. The repository notes that this interface is a subset and may not always match the canonical contracts.

The report therefore assumes that pools using these strategies behave consistently with the reviewed deployed contracts and interface. If the strategies are connected to another pool implementation or version, pool-dependent conclusions should be revalidated, particularly those related to profit and loss reporting.

Key Actors And Their Capabilities

The Pool Governor

The strategies do not maintain their own governor. Privileged governor functions check `pool().governor()`, making the governor of the consuming pool the governor of every strategy attached to it. The strategies enforce no timelock or quorum themselves, so any additional controls must exist at the pool level.

The governor has the broadest authority. It can upgrade the strategy implementation, report losses up to the strategy's full recorded debt through `reportLoss`, set the loss cap from zero to 100% through `setMaxLossBps`, change the deposit cap, and add or remove keepers. It can also change the fee collector that receives swept tokens and the swapper used for reward conversions.

Because the strategies are UUPS proxies, a compromised governor can replace the strategy implementation in a single transaction. `upgradeToAndCall` does verify that the governor remains unchanged across an upgrade, but this does not restrict what the new implementation itself can do. Since the role is inherited from the pool, compromising the pool governor also compromises every strategy attached to that pool.

Keepers

Keepers are maintained separately for each strategy and are added or removed by the governor. The account that initializes a strategy is automatically added as its first keeper.

Keepers perform routine strategy operations, including `rebalance`, `approveToken`, `claimAndSwapRewards`, and `swapToCollateral`. They choose the profit and loss bounds supplied to each rebalance and the slippage bounds used for swaps. On async strategies, they also manage redemptions through `requestRedeem`, `trackEpoch`, `untrackEpoch`, and `cancelRequest`. On `ConcreteAsyncManagedVault`, they additionally move principal through `depositToVault` and `withdrawFromVault`.

Keeper custody is limited: keeper operations do not allow collateral to be transferred to an arbitrary address, and `sweep` cannot transfer collateral or receipt tokens and sends eligible tokens only to the governor-configured fee collector.

However, keepers can materially affect accounting and operations. Incorrect epoch management can cause valid positions to disappear from accounting and be reported as losses. A keeper can also set approvals to zero, preventing new deposits, or choose weak slippage protection when swapping rewards. Keeper keys should therefore be protected as accounts with direct influence over strategy accounting and operations, even though they cannot directly transfer principal to themselves.

The Consuming Pool

The configured Pool is the only address permitted to invoke the strategy's `withdraw()` function. It also supplies the strategy's debt and excess-debt accounting and receives reported profits and repayments. Incorrect or malicious pool behavior could cause assets to be withdrawn unexpectedly or cause strategy reporting to be evaluated against incorrect debt values. The pool also provides the accounting values used by the strategy, including `excessDebt`, `creditLimit`, and `totalDebtOf`, and determines the strategy governor through `pool().governor()`.

The pool address is selected during initialization and cannot be changed through the current strategy interface. Its own administrative roles and accounting configuration are managed outside these contracts. Strategies should only be connected to verified Pool deployments whose governance and operational roles satisfy equivalent security requirements.

This trust is inherent to the architecture: the strategy manages capital on behalf of the pool. As a result, securing or reviewing the strategy alone is not sufficient; control over the pool also implies control over the strategy.

Concrete Vault Administrators

Authorized actors within the integrated Concrete vault may be able to upgrade the vault, pause operations, modify deposit or withdrawal limits, and change queue behavior. These actions can change the availability or reported value of strategy assets and may invalidate behavioral assumptions made by the integration. The strategy's configuration and assumptions should be reviewed following any Concrete vault upgrade or administrative change.

Concrete Vault Operators and Administrators

Concrete operators do not hold a role directly on the strategies, but they control external vault behavior that materially affects them. This includes enabling the redeem queue, closing and processing epochs, moving redemption requests between epochs, pausing the vault, and managing hooks. The Concrete vaults are also upgradeable proxies outside the scope of this audit.

The strategies rely on Concrete state and values such as `previewRedeem`, `maxWithdraw`, and epoch status when determining liquidity and position value. Concrete operator actions can therefore affect when strategy capital becomes available and how the strategy accounts for its position.

The main risk is delayed liquidity or incorrect accounting inputs rather than direct access to strategy funds. This external dependency is discussed further in the Operational Considerations section.

Non-Privileged Actors

Unprivileged addresses cannot directly modify strategy state; state-changing functions are restricted to the roles described above. Ordinary users interact with the consuming pool rather than directly with the strategies.

One intentional exception exists for Merkl rewards: during initialization, the strategy allows anyone to trigger reward claims on its behalf. Claimed rewards can only be sent to the strategy, and the governor can revoke this permission.

An ordinary pool deposit can also trigger conditions related to findings described elsewhere in this report, such as the unfunded-profit issue. This is a consequence of the strategy's accounting behavior rather than a privilege granted to the depositor.

Findings

VET-1

`_rebalance()` Sizes Reportable Profit Against The Credit Line, Which • Medium ⓘ Fixed
`reportEarning()` Does Not Permit

✓ Update

The development team bounded the reported profit and payback to the actual amount of collateral available in the contract. Addressed in `d9acb54`.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`, `test/mocks/VPoolMock.sol`

Description: `ConcreteVault._rebalance()` sizes the profit and payback it reports against a budget that includes the strategy's available credit line:

```
// contracts/strategies/concrete/ConcreteVault.sol:209-218
// Budget-aware sizing. `reportEarning` netts `(profit + payback)` against the strategy's
// credit line and only pulls the residual from idle, so profit is reportable without idle
// whenever the credit line covers it. [...]
uint256 _budget = _collateralHere + _pool.creditLimit(address(this));
_payback = Math.min(_excessDebt, _budget);
uint256 _reportableProfit = Math.min(_profit, _budget - _payback);
```

```
...
_pool.reportEarning(_profit, _loss, _payback);
```

The premise is incorrect. `YieldVault.reportEarning()` opens with an unconditional guard on the strategy's idle balance, roughly forty lines before any credit-line arithmetic runs:

```
// YieldVault.sol:340 (impl 0xf98F32D76d206ab4971be058Ef9A38CA4C21c514)
if (IERC20(asset()).balanceOf(_strategy) < (profit_ + payback_)) revert InsufficientBalance();
```

Whenever the credit line is open and profit exceeds the strategy's idle collateral, `rebalance()` therefore reverts outright — no profit or loss is reported, no payback is delivered, and for `ConcreteAsyncVault` the `_beforeReport()` epoch claim and `_afterReport()` redeem queueing roll back with it. The strategy is stuck until either governance closes the credit line by lowering `debtRatio`, or a pool-initiated withdrawal moves enough collateral into the strategy to cover the profit.

The condition is the expected steady state on a queue-active Concrete vault rather than an edge case. Yield accrues as share appreciation inside the Concrete vault; `_rebalance()`'s preparatory `_withdrawHere()` cannot convert it into idle while the redeem queue is live, and `_afterReport()` redeploys any remaining idle at the end of each cycle — so `_collateralHere()` sits near zero while `_profit` grows. `_availableCreditLimit()` returns non-zero whenever the strategy is below its debt ratio and the vault holds idle assets, which is true after any deposit into the vault. Four of the five vaults in scope run with the queue active.

No existing test can detect this. `test/mocks/VPoolMock.sol` implements the credit-line netting but omits the balance guard, and `test/Strategy.t.sol:81` etches that mock's bytecode over the pool address even in the fork fixtures that pass a real pool. `test/concrete/Concrete.t.sol:651 test_rebalance_reportsProfitAgainstCreditLine` then asserts the defective behaviour as correct, restating the same incorrect premise in its NatSpec, so the issue is pinned in place by a passing test.

Reproduction Test: [View test](#)

Recommendation: Bound the reported total by the collateral the strategy actually holds.

VET-2

Epoch Migration Can Hide Pending Redemptions, Causing the Strategy to Report a False Loss and Reduce Its Allocation

• Medium ⓘ Fixed

✓ Update

The strategy now tracks the number of shares it has queued and, before each report, verifies that Concrete still includes those shares in the tracked epochs. If any shares are missing, the report reverts with `RequestSharesChanged` rather than recording a loss. The keeper can then repair the bookkeeping by calling `trackEpoch` or `resyncRequests`. Addressed in: `16258e5`, `e896e35`, `5d07dcb`.

File(s) affected: `contracts/strategies/concrete/ConcreteAsyncVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`

Description: `ConcreteAsyncVault` tracks pending redemptions by epoch and includes their value when calculating the strategy's position. However, Concrete can move an unfunded redemption request to a later epoch without automatically updating the strategy.

After a request is moved, the strategy continues tracking the original epoch, where the migrated amount no longer appears. Because the corresponding shares are already escrowed by Concrete, they are also absent from the strategy's balance. The strategy therefore understates its position even though the redemption remains valid in the destination epoch.

On the next `rebalance()`, this accounting shortfall is interpreted as a loss. If it exceeds the configured loss cap, the rebalance reverts until the tracked epochs are corrected. Otherwise, the pool books a false loss, reducing its recorded debt and share price. Users redeeming before the accounting is corrected can consequently receive less than the economic value of their shares.

Tracking the destination epoch makes the position visible again, and the recovered value is subsequently reported as profit. However, the false loss also reduces the strategy's debt ratio, which the profit report does not restore. The strategy therefore remains underallocated until a maintainer manually restores the ratio, potentially leaving capital idle in the pool.

No underlying assets are lost; the reported loss results solely from the strategy temporarily losing track of a valid redemption after it moves between epochs.

Exploit Scenario:

1. A `ConcreteAsyncVault` submits a redemption request and records the corresponding Concrete epoch.
2. Concrete cannot fully fund that epoch and moves part of the request to a later epoch.
3. The strategy continues tracking only the original epoch, where the migrated portion is no longer visible, while the corresponding shares remain escrowed.
4. On the next `rebalance()`, the strategy interprets the resulting accounting shortfall as a loss.
5. If the loss is within the configured loss cap, the pool accepts it, reducing its recorded debt, share price, and the strategy's debt ratio.
6. Once the destination epoch is tracked, the missing position reappears and is reported as profit. The share price recovers, but the reduced debt ratio remains until a maintainer restores it.

Recommendation: Ensure the strategy can detect when a tracked redemption has been moved to another epoch instead of treating the missing request as having zero value.

One approach is to record the expected size of each redemption when it is submitted and reconcile it against the request reported by Concrete. If the request disappears or decreases before settlement, the strategy should treat this as an accounting mismatch rather than immediately recognizing the difference as a loss.

The recovery procedure for epoch desynchronization should also account for the strategy's debt ratio. If a false loss has already been reported, the debt ratio should be restored after correcting the tracked epochs, as the subsequent profit report does not restore it automatically.

VET-3

Enabling the Redeem Queue Can Turn Synchronous Withdrawals Into Untracked Requests, Causing False Losses

• Medium ⓘ

Fixed

✓ Update

The team added a check at initialization to `Concrete` and `ConcreteAsync` strategies so they are wired to the right kind of Concrete vault: `Concrete` now rejects any vault that exposes the redeem-queue interface and `ConcreteAsync` rejects any vault that does not. The only caveat is if Concrete later upgrades a vault to its async implementation and enables the queue, the issue would arise again until it is migrated to `ConcreteAsync`. This requires a Concrete-side upgrade and is fully recoverable. Addressed in: `66f3fdc`.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`

Description: `ConcreteVault` assumes withdrawals from its underlying Concrete vault settle synchronously. However, the Concrete vault's redeem queue can be enabled at runtime, and the strategy does not check whether it remains inactive.

When the queue is active, `withdraw()` does not transfer assets immediately. Instead, Concrete escrows the strategy's shares and creates a redemption request while returning the requested asset amount. Because the call does not revert, `ConcreteVault` treats the withdrawal as successful even though it received no assets.

The escrowed shares are no longer part of the strategy's balance, and `ConcreteVault` does not track pending redemption requests. The requested amount therefore disappears from the strategy's accounting and can be reported as a loss on the next `rebalance()`. If the loss exceeds the configured loss cap, the rebalance reverts instead.

However, the loss cap is enforced per report rather than cumulatively. If the untracked position is written off through several smaller losses, each can remain below the cap even though their combined value eventually equals the full escrowed amount. A zero loss cap would instead block any such loss report and preserve the pool's recorded claim until the accounting is corrected. Further withdrawals can continue creating untracked requests while the queue remains active.

No underlying assets are lost, as the shares remain escrowed by Concrete. However, recovering them requires migrating the strategy to `ConcreteAsyncVault` or otherwise processing the pending requests. If the affected epoch is still open, the request can be cancelled and the shares returned immediately; otherwise, the request must be tracked and processed. If a false loss was already reported, the strategy's debt ratio must also be restored separately.

Recommendation: Ensure `ConcreteVault` only performs synchronous withdrawals while the underlying Concrete vault's redeem queue is inactive.

Reject an underlying vault whose queue is already active during initialization, and make `_syncWithdrawable` return zero if the queue becomes active later. This prevents synchronous withdrawal attempts from creating untracked redemption requests. `ConcreteAsyncVault` already performs an equivalent queue check.

If the queue is enabled for a live strategy, migrate it to `ConcreteAsyncVault` so pending redemptions can be tracked and processed correctly. Monitor queue-status changes so this migration can be performed promptly, particularly while pending requests can still be cancelled.

Consider also emitting an event when a withdrawal is suppressed because the queue is active. Otherwise, the strategy can silently stop withdrawing, making the condition harder for operators to detect.

VET-4

A Zero Default Loss Cap Causes Any Shortfall to Block Rebalancing

• Low ⓘ

Fixed

✓ Update

The development team initialized `_maxLossBps` to one basis point by default. Addressed in `4f77d7a`

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`

Description: Concrete strategies use `maxLossBps` to limit the loss that can be reported during `rebalance()`. If the detected loss exceeds this limit, `_checkLossCap` reverts the entire transaction, preventing the strategy from reporting earnings, recording the loss, or returning collateral to the pool.

However, `maxLossBps` is not initialized and therefore defaults to zero until explicitly configured by the governor. With this default, any detected shortfall causes `rebalance()` to revert, including negligible differences caused by rounding when valuing the underlying position.

As a result, a strategy that is deployed without explicitly configuring `maxLossBps` can remain unable to rebalance until the governor updates the cap or reports the loss separately. This is particularly relevant to `ConcreteAsyncManagedVault`, where `rebalance()` is the mechanism used to keep the pool's recorded debt aligned with the value of the managed position.

The setter also permits `MAX_BPS`, effectively allowing the entire recorded debt to be reported as a loss. The behavior of both boundary values should therefore be considered explicitly when configuring the strategy.

Recommendation: Initialize `maxLossBps` to an appropriate non-zero value before the strategy becomes operational, either directly during initialization or as a required deployment step.

Document the semantics of the boundary values clearly: a value of zero prevents any detected loss from being reported through `rebalance()`, while `MAX_BPS` effectively permits losses up to the strategy's entire recorded debt.

The recovery procedure for losses exceeding the configured cap should also be documented. In particular, the governor can use `reportLoss` to record an accepted loss before allowing subsequent rebalances to proceed.

VET-5

A Failed Claim Causes the Strategy to Withdraw Too Little and Remain Above Its Deposit Cap

• Low ⓘ Fixed

✓ Update

The development team implemented the recommended fix in `ce35d4b`.

File(s) affected: `contracts/strategies/concrete/ConcreteAsyncVault.sol`

Description: `ConcreteAsyncVault` attempts to collect processed redemption requests at the start of every `rebalance()`. If a claim fails, the strategy emits `VaultActionSkipped` and continues, leaving the assets uncollected and the corresponding epoch tracked.

Later in the same rebalance, `_afterReport` may need to withdraw assets from the Concrete vault to bring the strategy's position back within the deposit cap. However, when the redeem queue is inactive, it reduces the required withdrawal by the amount that remains claimable (`L164–L171`):

```
uint256 _claimable = _claimableAssets();
if (_assetsNeeded > _claimable) {
    _withdrawHere(_assetsNeeded - _claimable);
    _idle = collateralToken().balanceOf(address(this));
}
```

This can cause the strategy to withdraw too little. For example, suppose the position is 300 assets above its deposit cap and 100 assets remain claimable because the earlier claim failed. `_afterReport` assumes those 100 assets will cover part of the required reduction and asks `_withdrawHere` for only 200. If the claim fails again, only 200 assets are removed and the strategy remains 100 above its cap.

The subtraction in `_afterReport` is unnecessary because `_withdrawHere` already retries the claim and accounts for the amount actually received (`L345–L354`):

```
function _withdrawHere(uint256 amount_) internal virtual override {
    uint256 _claimed = _tryClaimProcessedEpochs();
    if (_claimed >= amount_) return;
    IConcreteAsyncVault _vault = IConcreteAsyncVault(receiptToken());
    if (_vault.isQueueActive()) return;
    super._withdrawHere(amount_ - _claimed);
}
```

If `_afterReport` passed the full 300 assets to `_withdrawHere`, a successful 100-asset claim would leave 200 to withdraw. If the claim failed, `_withdrawHere` would instead withdraw the full 300. In both cases, the strategy would remove the required amount.

The current logic can also prevent later rebalances from correcting the problem. After the example above, the strategy is 100 above its cap and still has 100 claimable. On the next rebalance, `_assetsNeeded > _claimable` is false, so `_withdrawHere` is not called at all. If the claim continues to fail, the strategy can remain above its deposit cap across subsequent rebalances.

No assets are lost. The unclaimed assets remain owed to the strategy by Concrete, and the position returns within its cap once they are successfully collected. `ConcreteAsyncManagedVault` is not affected, as its `_rebalance` override does not run `_afterReport`.

Exploit Scenario:

1. A `ConcreteAsyncVault` has a deposit cap of 700 and a position worth 1,000, including 100 assets in a processed but unclaimed epoch.
2. The strategy attempts to claim the 100 assets at the start of `rebalance()`, but the claim fails.
3. `_afterReport` determines that 300 assets must be removed to bring the position down to the 700 cap.

- 4. Because 100 assets remain claimable, `_afterReport` calls `_withdrawHere(200)` instead of `_withdrawHere(300)`.
- 5. The claim inside `_withdrawHere` also fails, so the strategy withdraws only 200 and ends with a position worth 800.
- 6. On the next rebalance, the strategy is 100 above its cap and still has 100 claimable. The condition `_assetsNeeded > _claimable` is false, so no withdrawal is attempted.
- 7. The strategy remains above its deposit cap while the claim continues to fail.

Recommendation: Pass the full `_assetsNeeded` to `_withdrawHere` without subtracting `_claimableAssets()`:

```
    } else {
      _withdrawHere(_assetsNeeded);
      _idle = collateralToken().balanceOf(address(this));
    }
  }
```

`_withdrawHere` already retries the claim and subtracts only the assets it successfully receives before withdrawing the remainder. This ensures that the strategy removes the full required amount regardless of whether the claim succeeds or fails.

VET-6

`_tryWithdraw()` Does Not Apply The Vault's Minimum-Withdraw Floor To The Requested Amount

- Low ⓘ
- Fixed

✓

Update

The development team ensured that an amount no lower than the minimum withdrawal limit is passed to `_vault.withdraw()`. Addressed in `d001113`.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`

Description: `_syncWithdrawable()` reads the minimum withdrawal floor via `getWithdrawLimits()` and returns `0` when the *available* liquidity falls below it, but `_tryWithdraw()` then clamps the caller's request down to `Math.min(assets_, _syncWithdrawable())` without re-checking it against the floor. The gate therefore never tests the value actually sent to the vault, and any request smaller than `minWithdrawAmount` passes it. Concrete vaults reject it, and the `try/catch` swallows the revert into a `VaultActionSkipped` event, so nothing is withdrawn.

```
function _tryWithdraw(uint256 assets_) internal {
  assets_ = Math.min(assets_, _syncWithdrawable());
  if (assets_ == 0) return;

  IConcreteVault _vault = IConcreteVault(receiptToken());

  // solhint-disable-next-line no-empty-blocks
  try _vault.withdraw(assets_, address(this), address(this)) returns (uint256) {} catch (bytes memory _reason) {
    emit VaultActionSkipped(IERC4626.withdraw.selector, _reason);
  }
}
```

The consequence is a silent short-fill rather than a failed transaction. `instantLiquidity()` reports `idle + _syncWithdrawable()` and the consuming vault sizes user withdrawals from that figure, so when `Strategy.withdraw()` reaches `_tryWithdraw()` with a sub-floor amount the pool receives only the strategy's idle balance, under-delivering by up to `minWithdrawAmount` against a number the strategy itself published — the over-reporting that `Strategy.instantLiquidity()` documents as unsafe.

Severity is limited because the path is currently unreachable on most of the deployed Concrete vaults, which run with the redeem queue active — `ConcreteAsyncVault._syncWithdrawable()` short-circuits to `0` in that configuration — and because the shortfall is bounded by the floor.

Recommendation: A correct fix should ensure the amount passed to `_vault.withdraw()`, not merely the liquidity available to the strategy, satisfies the vault's `minWithdrawAmount`.

VET-7 Unbounded `_pendingEpochs` Set Is Iterated

- Low ⓘ
- Mitigated

i

Update

The issue is mitigated in three ways:

1. A processed epoch with nothing left to collect is pruned automatically on every claim.

2. Gas optimizations, including reducing the number of calls and operations per epoch.

3. `untrackEpoch()` remains as a manual workaround when needed.

Addressed in: `e896e35` , `5d07dcb` .

File(s) affected: `contracts/strategies/concrete/ConcreteAsyncVault.sol`

Description: `ConcreteAsyncVault` tracks queued redeem requests in an `EnumerableSet.UintSet` with no cap on its size. Entries are added by `requestRedeem()` , by `_afterReport()` 's automatic redeem sizing, and by `trackEpoch` ; they are removed only when `_tryClaimProcessedEpochs` successfully claims an epoch, when `cancelRequest()` clears a request, or by an explicit `untrackEpoch` . Four functions copy the entire set into memory with `.values()` and loop it, each iteration making one to three external calls into the vault:

1. `_claimableAssets()` , reached from `instantLiquidity()` .
2. `_queuedAssets()` , reached from `_getCollateralInProtocol()` and therefore from `tv1()` .
3. `_queuedShares()` , reached from `_toSizedRedeemShares()` during `_afterReport()` .
4. `_tryClaimProcessedEpochs()` , reached from `_beforeReport` and `_withdrawHere()` .

`instantLiquidity()` and `tv1()` are called by the consuming vault, once per strategy in its withdrawal queue, when sizing and servicing user withdrawals. The per-entry cost is also not constant: `_queuedAssets()` calls `previewRedeem()` for each pending epoch, and that call loops every strategy registered on the Concrete vault to preview yield and fees before returning.

Accumulation is slow in normal operation, but it is unbounded in principle, has no automatic ceiling, and the only manual prune available to a keeper is `untrackEpoch` , which drops the entry's value from `_queuedAssets()` .

Recommendation: Ensure the gas cost of `instantLiquidity()` and `tv1()` cannot grow without limit as tracked epochs accumulate. Rejecting a new entry once a specified ceiling is reached is the most direct option.

VET-8

Managed-vault deposit caps do not bound ongoing vault exposure

• **Low** ⓘ **Acknowledged**

Update

The development team acknowledged the issue and provided the following explanation:

Entry-only is the intended semantics. In the managed variant principal moves solely by keeper action, so an automatic unwind would contradict the contract's defining rule — holding a position through a depeg unless the keeper deliberately exits. The cap is documented as an entry limit, with keeper monitoring to correct over-allocation. Quantstamp's related note was adopted in part: `_checkRequestedShares()` now runs at the top of the managed `_rebalance`, so the epoch invariant is enforced there too; the claim half of `_beforeReport` stays out because it moves assets.

File(s) affected: `contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`

Description: `ConcreteAsyncManagedVault.depositToVault()` routes deposits through `_tryDeposit()` , which prevents new deposits from exceeding the configured `depositCap` . However, the managed strategy overrides `_rebalance()` and does not invoke the inherited `_afterReport()` logic that unwinds positions exceeding the cap.

Consequently, the cap is enforced only when assets enter the Concrete vault. The deployed position can remain above the cap indefinitely if its value increases through yield or governance lowers the cap below the existing position. Because the managed strategy moves principal only through keeper actions, restoring compliance requires the keeper to explicitly request and complete a withdrawal. If `depositCap` is intended to limit the strategy's ongoing exposure to the Concrete vault, the configured risk limit is therefore not reliably enforced.

Recommendation: Either add an explicit mechanism that initiates withdrawal of collateral exceeding the cap, or document the value as an entry-only deposit limit and require keeper monitoring and unwinding when the position exceeds it.

VET-9

Missing Post-Deployment Configuration Can Leave Strategies Silently Non-Operational

• **Informational** ⓘ **Acknowledged**

Update

The development team acknowledged the issue and provided the following explanation:

Operational, not a contract defect. Approvals, loss cap, deposit cap, keeper registration and debt ratio are owned by the deploy scripts and the post-deploy checklist, which run and verify them before capital is allocated. Enforcing the sequence on-chain would add an initialization gate to every upgrade path without removing the need for the same off-chain verification.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol` ,
`contracts/strategies/concrete/ConcreteAsyncVault.sol` ,
`contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`

Description: The Concrete strategies require several privileged configuration steps after deployment, but nothing ensures they are completed before the strategies receive capital.

In particular, token approvals are not granted during initialization and must be set through the keeper-gated `approveToken` . Until then, deposits into Concrete fail. Because deposits are best-effort, the rebalance still succeeds while the deposit is skipped. The failure is reported through `VaultActionSkipped` , but tooling that checks only transaction status may incorrectly assume the strategy is operating normally.

Other required setup includes the loss cap, deposit cap where applicable, keeper registration, and the pool-side debt ratio. At the audited commit, no deployment scripts define or verify these steps. The loss cap behavior is reported separately.

Recommendation: Consider adding an initialization method that performs or verifies the required setup on-chain before the strategy can be used. Alternatively, create a well-tested deployment process that ensures all required configuration is completed before the strategy is used in production.

At minimum, this should cover `approveToken` , `setMaxLossBps` , `setDepositCap` where applicable, keeper registration, and the pool-side debt ratio. The deployment process should also perform a small rebalance and verify that the expected deposit succeeds.

VET-10

Missing Events Make Principal Movements and Skipped Withdrawals Harder to Monitor

• Informational ⓘ Fixed

✓ Update

The team refactored the code to emit `RedeemRequested(epochID, shares)` whenever a redeem request is booked through either entry point, and `VaultActionUnsize(selector, requested)` whenever a deposit, withdrawal, or redemption is sized to zero. Addressed in: `8cc34d3` , `e896e35` , `5d07dcb` .

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol` ,
`contracts/strategies/concrete/ConcreteAsyncVault.sol`

Description: Some actions that affect strategy principal are not directly observable through events. In particular, `requestRedeem` starts unwinding principal without emitting an event. Similarly, when a withdrawal is reduced to zero because no assets can currently be withdrawn, the function returns without emitting `VaultActionSkipped` or another signal.

Best-effort calls that fail do emit `VaultActionSkipped` , but the failed action must be inferred from the selector carried by the event.

As a result, monitoring systems cannot rely on the strategy's event stream alone to determine when principal starts being withdrawn or when an intended action is silently skipped. This can make operational issues harder to detect and respond to promptly.

Recommendation: Consider emitting events when important actions are initiated or skipped. In particular, emit an event when a redemption request is submitted, and when an intended withdrawal is reduced to zero because no assets are currently withdrawable. The events should include enough context to identify the affected amount and action.

Monitoring should also account for `VaultActionSkipped` , the strategy's epoch events, Concrete's queue and request-migration events, and non-zero losses reported by the pool.

VET-11

`_syncWithdrawable()` Comment Overstates The Pre-v1.5.0 `maxWithdraw` Over-Report

• Informational ⓘ Fixed

✓ Update

The comment above `_syncWithdrawable()` has been corrected in `8cc34d3` .

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`

Description: The comment above `_syncWithdrawable()` states that pre-v1.5.0 Concrete implementations size it off the vault's raw token balance and over report by exactly that figure , the figure being `pastEpochsUnclaimedAssets` . That amount is an upper bound on the discrepancy, not the discrepancy itself.

What `maxWithdraw()` advertises and what the vault will actually pay diverge only when the vault cannot fully cover the requested amount. Whenever it can — whether from its own token balance or by pulling from the positions behind it — the two figures agree, and there is no over-report at all, however much the vault owes to processed epochs. The size of the gap is therefore a function of the vault's available liquidity rather than of its outstanding obligations, and it reaches `pastEpochsUnclaimedAssets` only when the vault is fully liquidity-constrained.

Of the two vaults the comment names, that is true of `srRoyUSDC` alone, which can currently source almost nothing beyond its token balance. `ctDeltaWeETH` can cover its entire position and owes nothing to processed epochs, so no netting occurs there at all. The strategy's behaviour is correct in both cases, since subtracting the full figure unconditionally is exact for a liquidity-constrained vault and conservative otherwise, so this is a documentation issue only.

Recommendation: Reword to say the pre-v1.5.0 implementations over-report by *up to* `pastEpochsUnclaimedAssets`, and note that the size of the gap depends on the vault's available liquidity rather than on what it owes alone.

VET-12

`_approveToken()` Couples The Pool And End-Protocol Allowances To A Single Parameter

• **Informational** ⓘ **Acknowledged**

i Update

The development team acknowledged the issue and provided the following explanation:

Out of scope for this engagement. The coupling lives in Strategy and ERC4626Vault, shared base contracts behind every ERC-4626 strategy in the repository; splitting the allowances would touch each one and require regression coverage well beyond the Concrete family. The containment concern is accepted and tracked separately, to be addressed repo-wide rather than in an audit-remediation branch.

File(s) affected: `contracts/strategies/ERC4626Vault.sol`, `contracts/strategies/Strategy.sol`

Description: `approveToken(amount_)` grants the same value to two unrelated spenders. `Strategy._approveToken()` approves the pool, and the `ERC4626Vault` override then approves the receipt token for the same amount:

```
function _approveToken(uint256 amount_) internal virtual override {
    super._approveToken(amount_); // pool
    collateralToken().forceApprove(receiptToken(), amount_); // Concrete vault
}
```

Both allowances are necessary — the pool's is consumed by `reportEarning()`, which pulls the residual with `safeTransferFrom(_strategy, ...)`, and the receipt token's by the Concrete vault's `deposit()`. The problem is that a single parameter drives both despite them serving different purposes against counterparties with very different trust profiles: the protocol's own pool on one side, a third-party upgradeable proxy on the other. They cannot be sized independently, and more importantly, they cannot be revoked independently. Calling `approveToken(0)` in response to an incident at the end protocol would simultaneously strip the pool's ability to collect profit and payback, partially disabling `reportEarning` — which removes the obvious containment lever at exactly the moment it is wanted.

The practical effect is amplified by the amount actually used, which is `type(uint256).max` throughout. The Concrete vault therefore holds unlimited spending rights over the strategy's entire collateral balance rather than only what has been deposited. That distinction is meaningful here because these strategies routinely hold collateral they have deliberately kept out of the vault: reserved for the next payback, or idle because deposits are disabled, the deposit cap is reached, or a hook is refusing them. Principal already inside the vault is exposed to that vault's upgrade path regardless, but the allowance extends the same exposure to collateral the strategy chose not to deposit.

Recommendation: Allow the end-protocol allowance to be set and revoked independently of the pool allowance.

Auditor Suggestions

S1 Provide an atomic helper for replacing moved epochs

Acknowledged

i Update

The development team acknowledged the suggestion and provided the following explanation:

Reviewed and not adopted; the change would not have improved the property it targeted.

File(s) affected: `contracts/strategies/concrete/ConcreteAsyncVault.sol`

Description: `trackEpoch()` and `untrackEpoch()` are separate keeper operations, although their documented primary use case is updating the strategy's bookkeeping after Concrete moves a withdrawal request to another epoch. Requiring two transactions increases operational complexity and can leave stale or incomplete epoch tracking if only one operation succeeds.

Recommendation: Add an atomic `updateEpoch(oldEpochID, newEpochID)` function that verifies the old epoch is tracked, the new epoch contains a strategy request, and the old epoch no longer contains one before replacing it.

S2 Rename the strategy contracts to distinguish them from Concrete vaults

Fixed

✓

Update

The contracts have been renamed as follows:

1. ConcreteVault → Concrete

2. ConcreteAsyncVault → ConcreteAsync

3. ConcreteAsyncManagedVault → ConcreteAsyncManaged

Addressed in `8cc34d3` and `ac6ba13`.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`

Description: The contracts named `ConcreteVault`, `ConcreteAsyncVault`, and `ConcreteAsyncManagedVault` are strategies that deposit into separate vaults operated by the Concrete protocol. Referring to both the strategy and its external integration as a “Concrete vault” makes the direction of interactions and ownership of assets difficult to follow, increasing the likelihood of mistakes during development, review, and incident response.

Recommendation: Consider renaming the contracts to `ConcreteStrategy`, `ConcreteAsyncStrategy`, and `ConcreteAsyncManagedStrategy`, respectively. If no existing proxy uses the current storage layout, rename the corresponding ERC-7201 namespace strings as well. Otherwise, preserve the existing namespace strings and calculated storage slots to maintain upgrade compatibility.

S3 Configure Concrete vault capabilities explicitly

Acknowledged

i

Update

The development team acknowledged the suggestion and provided the following explanation:

Capability configuration. Deliberately held for a later change rather than folded into audit remediation.

File(s) affected: `contracts/strategies/concrete/ConcreteVault.sol`,
`contracts/strategies/concrete/ConcreteAsyncVault.sol`

Description: The strategies dynamically infer safety-critical behavior from optional selectors and implementation versions:

1. `_isVaultPaused()` assumes that `paused() == true` makes both synchronous withdrawals and claims from processed epochs unavailable, although neither the selector nor these pause semantics are specified by `ConcreteVault`.

2. `_nonWithdrawableAssets()` assumes that an `ID()` corresponding to version 1.5.0 or later means `maxWithdraw()` already excludes assets owed to processed epochs.

Although these assumptions appear valid for the reviewed deployments, they are not enforced by the integration interface and may not hold across different flavours, backports, or future Concrete proxy upgrades. An incorrect inference could cause `instantLiquidity()` and withdrawal sizing to over- or under-report available assets.

Recommendation: Store the relevant vault capabilities as explicit per-deployment configuration, including whether pausing blocks synchronous withdrawals, whether it blocks epoch claims, and whether `maxWithdraw()` already nets unclaimed epoch assets. Continue reading the live pause status when applicable, but interpret it according to the configured behavior rather than inferred semantics. Allow governance to update the configuration following a reviewed Concrete vault upgrade, and avoid deriving accounting behavior solely from `ID()`.

Definitions

- High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.

- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/blogpriv/vesper-strategies`

- `002...a31 ./contracts/strategies/concrete/ConcreteAsyncManagedVault.sol`
- `e0b...8e7 ./contracts/strategies/concrete/ConcreteAsyncVault.sol`
- `cfb...643 ./contracts/strategies/concrete/ConcreteVault.sol`

Vulnerability Reproduction Tests

These tests are designed to be copied into the project's existing test suite to reproduce the associated findings. They can then be retained as regression tests to verify that the findings have been remediated and do not recur.

VET-1 `_rebalance()` Sizes Reportable Profit Against The Credit Line, Which `reportEarning()` Does Not Permit

Reproduction test | Language: Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

/**
 * Proof of concept: ConcreteVault._rebalance sizes reportable profit against the strategy's
 * credit line, but YieldVault.reportEarning requires the strategy to hold profit + payback
 * as idle collateral before any credit-line netting happens.
 *
 * YieldVault.sol:340
 *     if (IERC20(asset()).balanceOf(_strategy) < (profit_ + payback_)) revert InsufficientBalance();
 *
 * The netting further down (_totalPayback < _creditLine ? transfer : transferFrom) only decides
 * the direction of the transfer once that guard has already passed; it does not relieve it.
 *
 * test/mocks/VPoolMock.sol models the netting and omits the guard, and every test in the suite
 * runs against that mock -- test/Strategy.t.sol:81 etches its bytecode over the pool address even
 * in the fork fixtures -- so no existing test can observe the constraint.
 *
 * Two independent demonstrations:
 * 1. Concrete_ReportEarning_LiveVault_PoC - the constraint exists on deployed bytecode.
 * 2. Concrete_ReportEarning_Strategy_PoC - _rebalance produces a call that violates it.
 *
 * Run:
 *     ETHEREUM_FORK_NODE_URL=<rpc> forge test --match-path test/concrete/ConcreteReportEarning.poc.t.sol -vv
 */

import "forge-std/Test.sol";
```

```

import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

import {ConcreteAsyncVault} from "contracts/strategies/concrete/ConcreteAsyncVault.sol";
import {ERC4626Vault} from "contracts/strategies/ERC4626Vault.sol";

import {MockConcreteAsyncVault} from "test/mocks/MockConcreteAsyncVault.sol";
import {MasterOracleMock} from "test/mocks/MasterOracleMock.sol";
import {SwapperMock} from "test/mocks/SwapperMock.sol";
import {TestERC20} from "test/mocks/TestERC20.sol";
import {VPoolMock} from "test/mocks/VPoolMock.sol";

interface IYieldVaultPoC {
    function asset() external view returns (address);

    function creditLimit(address strategy_) external view returns (uint256);

    function excessDebt(address strategy_) external view returns (uint256);

    function getWithdrawQueue() external view returns (address[] memory);

    function reportEarning(uint256 profit_, uint256 loss_, uint256 payback_) external;
}

/*//////////////////////////////////////
    1 - THE CONSTRAINT, ON DEPLOYED BYTECODE
////////////////////////////////////*/

/// @dev Calls the live `vwvUSDC` YieldVault -- the same pool `test/concrete/*.ethereum.t.sol` names
///      as the Concrete fixtures' pool -- from one of its own registered strategies, reporting a
///      profit that sits strictly inside `idle + creditLimit`. That is exactly the `_budget`
///      `ConcreteVault._rebalance` computes, and the vault rejects it.
contract Concrete_ReportEarning_LiveVault_PoC is Test {
    /// @dev vwvUSDC. Shares implementation 0xf98F32D76d206ab4971be058Ef9A38CA4C21c514 with
    ///      vaUSDWhitelistedVault, so the result holds for the USDT fixture too.
    address internal constant VAULT = 0x3aEd4dd0b5ba616156Fa352274670c18D2Ec2A81;

    /// @dev A strategy registered on that vault which holds no idle collateral.
    address internal constant STRATEGY = 0x90D8BF785B8e77925Eb5EC6Cc65B9c07584FaaEd;

    /// @dev Pinned so the balances below are stable. Unpin to check against head.
    uint256 internal constant FORK_BLOCK = 25854433;

    /// @dev Mirrors `YieldVault.InsufficientBalance`.
    error InsufficientBalance();

    function setUp() public {
        vm.createSelectFork("ethereum", FORK_BLOCK);
    }

    function test_liveVault_rejectsProfitTheCreditLineWouldCover() public {
        IERC20 _asset = IERC20(IYieldVaultPoC(VAULT).asset());

        uint256 _idle = _asset.balanceOf(STRATEGY);
        uint256 _creditLimit = IYieldVaultPoC(VAULT).creditLimit(STRATEGY);
        uint256 _excessDebt = IYieldVaultPoC(VAULT).excessDebt(STRATEGY);

        assertEq(_excessDebt, 0, "precondition: no excess debt, so the whole budget goes to profit");
        assertGt(_creditLimit, 0, "precondition: the credit line is open");

        // The smallest profit that exceeds idle. `ConcreteVault._rebalance` would report this,
        // because its budget is `_collateralHere + creditLimit`, not `_collateralHere`.
        uint256 _profit = _idle + 1;
        assertle(_profit, _idle + _creditLimit, "profit sits inside the budget the strategy computes");

        vm.prank(STRATEGY);
        vm.expectRevert(InsufficientBalance.selector);
        IYieldVaultPoC(VAULT).reportEarning(_profit, 0, 0);

        // Control: the same call with profit capped at idle is accepted, so the guard keys purely

```



```

        // on `balanceOf < profit + payback` and is not relieved by the credit line.
        vm.prank(STRATEGY);
        IYieldVaultPoC(VAULT).reportEarning(_idle, 0, 0);
    }
}

/*//////////////////////////////////////
    2 - THE STRATEGY PRODUCES SUCH A CALL
////////////////////////////////////*/

/// @notice Minimal pool faithful to `YieldVault.reportEarning`. Identical in shape to
///         `test/mocks/VPoolMock.sol` except for the one guard marked below, which `VPoolMock`
///         does not implement and which is therefore invisible to the entire existing suite.
contract FaithfulYieldVaultPool {
    using SafeERC20 for IERC20;

    error InsufficientBalance();
    error IncorrectPayback(uint256 payback, uint256 expectedPayback);
    error ProfitAndLossCannotBeReportedTogether();

    IERC20 public immutable token;
    address public governor;

    uint256 public targetDebt;
    uint256 public currentDebt;
    uint256 public reportedProfit;
    uint256 public reportedLoss;

    constructor(IERC20 token_, address governor_) {
        token = token_;
        governor = governor_;
    }

    function setDebt(uint256 target_, uint256 current_) external {
        targetDebt = target_;
        currentDebt = current_;
    }

    function totalDebtOf(address) external view returns (uint256) {
        return currentDebt;
    }

    function excessDebt(address) public view returns (uint256) {
        return currentDebt > targetDebt ? currentDebt - targetDebt : 0;
    }

    /// @dev Mirrors `YieldVault._availableCreditLimit`, including its `min(_assetsInVault(), ...)`
    function creditLimit(address) public view returns (uint256) {
        uint256 _available = targetDebt > currentDebt ? targetDebt - currentDebt : 0;
        return Math.min(token.balanceOf(address(this)), _available);
    }

    function reportEarning(uint256 profit_, uint256 loss_, uint256 payback_) external {
        address _strategy = msg.sender;
        if (profit_ > 0 && loss_ > 0) revert ProfitAndLossCannotBeReportedTogether();

        // -----
        // YieldVault.sol:340 - the guard `VPoolMock` omits.
        // -----
        if (token.balanceOf(_strategy) < (profit_ + payback_)) revert InsufficientBalance();

        if (loss_ > 0) {
            reportedLoss += loss_;
            currentDebt = currentDebt > loss_ ? currentDebt - loss_ : 0;
        }

        uint256 _expectedPayback = excessDebt(_strategy);
        if (profit_ > 0 && payback_ < _expectedPayback) revert IncorrectPayback(payback_,
            _expectedPayback);

        uint256 _actualPayback = Math.min(_expectedPayback, payback_);
        currentDebt -= _actualPayback;
    }
}

```

```

        reportedProfit += profit_;

        uint256 _creditLine = creditLimit(_strategy);
        currentDebt += _creditLine;

        // The netting the strategy's comment reasons from. It runs only after the guard above.
        uint256 _totalPayback = profit_ + _actualPayback;
        if (_totalPayback < _creditLine) {
            token.safeTransfer(_strategy, _creditLine - _totalPayback);
        } else if (_totalPayback > _creditLine) {
            token.safeTransferFrom(_strategy, address(this), _totalPayback - _creditLine);
        }
    }
}

contract Concrete_ReportEarning_Strategy_PoC is Test {
    uint256 internal constant MAX_BPS = 100_00;
    uint256 internal constant SEED = 1_000e18;
    uint256 internal constant YIELD = 50e18;

    address internal constant MERKL = 0x3Ef3D8bA38EBe18DB133cEc108f4D14CE00Dd9Ae;

    TestERC20 internal asset;
    MockConcreteAsyncVault internal vault;
    FaithfulYieldVaultPool internal pool;
    ConcreteAsyncVault internal strategy;

    address internal governor = makeAddr("governor");

    function setUp() public {
        asset = new TestERC20("Mock USD", "mUSD");
        vault = new MockConcreteAsyncVault(address(asset), 18);
        pool = new FaithfulYieldVaultPool(IERC20(address(asset)), governor);

        vm.mockCall(MERKL, abi.encodeWithSignature("toggleOperator(address,address)"), abi.encode());

        bytes memory _initData = abi.encodeWithSelector(
            ERC4626Vault.initialize.selector,
            address(pool),
            address(new SwapperMock(new MasterOracleMock())),
            address(vault),
            "Concrete-PoC"
        );
        strategy = ConcreteAsyncVault(address(new ERC1967Proxy(address(new ConcreteAsyncVault()),
            _initData)));
        strategy.approveToken(type(uint256).max);

        vm.prank(governor);
        strategy.setMaxLossBps(uint16(MAX_BPS));

        // Draw the credit line down into the Concrete vault. Profit is zero on this first report,
        // so it lands cleanly and leaves the strategy with its whole position deployed.
        asset.mint(address(pool), SEED);
        pool.setDebt({target_: SEED, current_: 0});
        strategy.rebalance(0, type(uint256).max);

        assertEq(asset.balanceOf(address(strategy)), 0, "setup: strategy holds no idle collateral");
        assertApproxEqAbs(strategy.tvl(), SEED, 2, "setup: position deployed into the Concrete vault");
    }

    /// @dev The failing case. Yield accrues as share appreciation inside the Concrete vault, so the
    ///       strategy carries profit with zero idle. The redeem queue is live, so `_rebalance`'s
    ///       `_withdrawHere` cannot convert that appreciation into idle. `_budget` then includes the
    ///       open credit line, the strategy reports profit it does not hold, and the pool rejects it.
    function test_rebalance_revertsReportingProfitAgainstAnOpenCreditLine() public {
        _accrueYieldInsideTheVault();
        _openCreditLine(100e18);

        uint256 _profit = strategy.tvl() - pool.totalDebtOf(address(strategy));
        assertGt(_profit, 0, "there is profit to report");
        assertEq(asset.balanceOf(address(strategy)), 0, "and none of it is idle");
        assertGt(pool.creditLimit(address(strategy)), _profit, "the credit line appears to cover it");
    }
}

```

```

vm.expectRevert(FaithfulYieldVaultPool.InsufficientBalance.selector);
strategy.rebalance(0, type(uint256).max);
}

/// @dev The control. Same position, same profit, same zero idle -- only the credit line is
///      closed, so `_budget` collapses to `_collateralHere` and the strategy defers the profit
///      instead of over-reporting it. This isolates `creditLimit` in `_budget` as the cause.
function test_rebalance_succeedsWhenTheCreditLineIsClosed() public {
    _accrueYieldInsideTheVault();

    uint256 _debt = pool.totalDebtOf(address(strategy));
    pool.setDebt({target_: _debt, current_: _debt});
    assertEq(pool.creditLimit(address(strategy)), 0, "credit line closed");

    strategy.rebalance(0, type(uint256).max);

    assertEq(pool.reportedProfit(), 0, "profit is carried, not reported, when idle cannot cover it");
}

/// @dev And the same shape via `ERC4626Vault._rebalance`'s idle-bounded sizing would have been
///      accepted: `payback + profit <= _collateralHere` is exactly the pool's precondition.
function test_poolAcceptsProfitBoundedByIdle() public {
    _accrueYieldInsideTheVault();
    _openCreditLine(100e18);

    uint256 _profit = strategy.tvl() - pool.totalDebtOf(address(strategy));
    deal(address(asset), address(strategy), _profit);

    vm.prank(address(strategy));
    pool.reportEarning(_profit, 0, 0);

    assertEq(pool.reportedProfit(), _profit, "identical report succeeds once idle covers it");
}

/// @dev Why the suite is green. The identical scenario, run against the repo's own
///      `VPoolMock`, reports the profit and passes -- because that mock implements the
///      credit-line netting and omits the balance guard. Every existing test, unit and fork
///      alike, runs against this bytecode: `test/Strategy.t.sol:81` etches it over the pool
///      address even when a real pool is passed to `_initStrategy`.
function test_existingMockAcceptsWhatTheRealVaultRejects() public {
    VPoolMock _mockPool = new VPoolMock(address(asset));
    _mockPool.setGovernor(governor);

    MockConcreteAsyncVault _vault = new MockConcreteAsyncVault(address(asset), 18);
    bytes memory _initData = abi.encodeWithSelector(
        ERC4626Vault.initialize.selector,
        address(_mockPool),
        address(new SwapperMock(new MasterOracleMock())),
        address(_vault),
        "Concrete-PoC-MockPool"
    );
    ConcreteAsyncVault _strategy = ConcreteAsyncVault(
        address(new ERC1967Proxy(address(new ConcreteAsyncVault()), _initData))
    );
    _strategy.approveToken(type(uint256).max);
    vm.prank(governor);
    _strategy.setMaxLossBps(uint16(MAX_BPS));

    asset.mint(address(_mockPool), SEED);
    _mockPool.updateDebtOfStrategy({target_: SEED, latest_: 0});
    _strategy.rebalance(0, type(uint256).max);

    asset.mint(address(this), YIELD);
    asset.approve(address(_vault), YIELD);
    _vault.addYield(YIELD);

    uint256 _debt = _mockPool.totalDebtOf(address(_strategy));
    asset.mint(address(_mockPool), 100e18);
    _mockPool.updateDebtOfStrategy({target_: _debt + 100e18, latest_: _debt});

    assertEq(asset.balanceOf(address(_strategy)), 0, "same position: no idle collateral");
}

```

```

        // No revert, and the profit is booked -- against a pool that would have refused it.
        _strategy.rebalance(0, type(uint256).max);
        assertGt(_mockPool.strategyTotalProfit(), 0, "the mock accepts profit the strategy does not
hold");
    }

    /*//////////////////////////////////////
                                   HELPERS
    //////////////////////////////////*/

    /// @dev Lift the Concrete vault's share price without minting shares.
    function _accrueYieldInsideTheVault() private {
        asset.mint(address(this), YIELD);
        asset.approve(address(vault), YIELD);
        vault.addYield(YIELD);
    }

    /// @dev Reopen the credit line: the pool holds idle and the strategy sits below its target.
    function _openCreditLine(uint256 amount_) private {
        uint256 _debt = pool.totalDebtOf(address(strategy));
        asset.mint(address(pool), amount_);
        pool.setDebt({target_: _debt + amount_, current_: _debt});
    }
}

```

Test Suite Results

The tests were run using `forge test --match-path "test/concrete/*.t.sol"`. The test suite is thorough, includes both unit and integration tests, and covers a variety of scenarios for each vault integration. The branch coverage is also relatively high at `91.5 %`.

```

Ran 17 tests for test/concrete/ConcreteAsyncManagedVault.t.sol:ConcreteAsyncManagedVault_Test
[PASS] test_depositToVault_isBoundByTheCap() (gas: 304959)
[PASS] test_depositToVault_pushesIdleIntoTheVault() (gas: 271969)
[PASS] test_depositToVault_revertsForNonKeeper() (gas: 159962)
[PASS] test_depositToVault_skipsWhenAlreadyAtCap() (gas: 309742)
[PASS] test_instantLiquidity_excludesClaimable() (gas: 445650)
[PASS] test_instantLiquidity_isIdleOnly() (gas: 272242)
[PASS] test_rebalance_isInertOnADeployedPosition() (gas: 289209)
[PASS] test_rebalance_leavesEverythingIdleAndDeploysNothing() (gas: 162512)
[PASS] test_rebalance_paysBackFromIdleOnly() (gas: 296566)
[PASS] test_rebalance_profitIsDeferredUntilRealizedToIdle() (gas: 398815)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 284919)
[PASS] test_requestRedeem_escrowsSharesAndTracksTheEpoch() (gas: 403478)
[PASS] test_tvl_countsIdlePlusVaultPosition() (gas: 433170)
[PASS] test_userWithdraw_servesIdleAndNeverTouchesTheVault() (gas: 289734)
[PASS] test_withdrawFromVault_claimsProcessedEpochsIntoIdle() (gas: 445386)
[PASS] test_withdrawFromVault_revertsForNonKeeper() (gas: 159954)
[PASS] test_withdrawFromVault_syncWithdrawsWhenQueueOff() (gas: 308351)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 10.32ms (3.55ms CPU time)

```

```

Ran 19 tests for test/concrete/ConcreteDepositCap.t.sol:ConcreteAsyncVault_QueueOff_DepositCap_Test
[PASS] test_atTheCap_rebalanceIsIdempotent() (gas: 349556)
[PASS] test_atTheCap_yieldNeverGrowsThePosition() (gas: 465860)
[PASS] test_capAboveAllocation_isInert() (gas: 295822)
[PASS] test_capHeadroomBelowMinDeposit_staysUnderTheCap() (gas: 376097)
[PASS] test_capHolds_whenPoolKeepsPushingCredit() (gas: 355066)
[PASS] test_cappedAllocation_deploysUpToTheCapAndParksTheRest() (gas: 325221)
[PASS] test_depositCap_defaultsToUncapped() (gas: 15980)
[PASS] test_lossAtTheCap_reopensHeadroomAndRedeploysIdle() (gas: 439132)
[PASS] test_loweringTheCap_convergesWithAnOpenCreditLine() (gas: 418045)
[PASS] test_loweringTheCap_doesNotDisturbProfitAccounting() (gas: 479024)
[PASS] test_loweringTheCap_unwindsTheVaultPosition() (gas: 384552)
[PASS] test_poolWithdraw_reopensCapHeadroomForTheNextDeposit() (gas: 437877)
[PASS] test_revertingVaultWithdraw_degradesToSkip() (gas: 419810)
[PASS] test_setDepositCap_revertsForNonGovernor() (gas: 21136)
[PASS] test_setDepositCap_updatesAndEmits() (gas: 59785)
[PASS] test_uncapped_afterResettingCapToZero() (gas: 325253)

```



```
[PASS] test_uncapped_deploysEverythingAndNeverRepays() (gas: 258592)
[PASS] test_unreportedProfit_isReservedIdleAndNotSweptBack() (gas: 566007)
[PASS] test_vaultCapTighterThanStrategyCap_wins() (gas: 325615)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 11.92ms (5.42ms CPU time)
```

```
Ran 8 tests for test/concrete/ConcreteVault.unit.t.sol:ConcreteVault_Unit_Test
[PASS] test_deposit_skippedWhenPreviewMintsZeroShares() (gas: 283871)
[PASS] test_instantLiquidity_degradesToIdleOnStaleAccounting() (gas: 315829)
[PASS] test_instantLiquidity_ignoresEpochAccounting() (gas: 297139)
[PASS] test_instantLiquidity_tracksTheWithdrawableBand() (gas: 299923)
[PASS] test_queueActive_strandsPositionUnrecoverably() (gas: 406153)
[PASS] test_rebalanceAndWithdrawSettleInLine() (gas: 358686)
[PASS] test_tvlAndRebalance_stillRevertOnStaleAccounting() (gas: 293013)
[PASS] test_withdraw_survivesStaleAccounting() (gas: 323951)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.02ms (1.30ms CPU time)
```

```
Ran 19 tests for test/concrete/ConcreteDepositCap.t.sol:ConcreteVault_DepositCap_Test
[PASS] test_atTheCap_rebalanceIsIdempotent() (gas: 333768)
[PASS] test_atTheCap_yieldNeverGrowsThePosition() (gas: 441768)
[PASS] test_capAboveAllocation_isInert() (gas: 286928)
[PASS] test_capHeadroomBelowMinDeposit_staysUnderTheCap() (gas: 360309)
[PASS] test_capHolds_whenPoolKeepsPushingCredit() (gas: 339278)
[PASS] test_cappedAllocation_deploysUpToTheCapAndParksTheRest() (gas: 315015)
[PASS] test_depositCap_defaultsToUncapped() (gas: 15936)
[PASS] test_lossAtTheCap_reopensHeadroomAndRedeploysIdle() (gas: 423341)
[PASS] test_loweringTheCap_convergesWithAnOpenCreditLine() (gas: 387059)
[PASS] test_loweringTheCap_doesNotDisturbProfitAccounting() (gas: 451875)
[PASS] test_loweringTheCap_unwindsTheVaultPosition() (gas: 361746)
[PASS] test_poolWithdraw_reopensCapHeadroomForTheNextDeposit() (gas: 416422)
[PASS] test_revertingVaultWithdraw_degradesToSkip() (gas: 383086)
[PASS] test_setDepositCap_revertsForNonGovernor() (gas: 21136)
[PASS] test_setDepositCap_updatesAndEmits() (gas: 59697)
[PASS] test_uncapped_afterResettingCapToZero() (gas: 310751)
[PASS] test_uncapped_deploysEverythingAndNeverRepays() (gas: 252296)
[PASS] test_unreportedProfit_isReservedIdleAndNotSweptBack() (gas: 535732)
[PASS] test_vaultCapTighterThanStrategyCap_wins() (gas: 315409)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 4.24ms (3.45ms CPU time)
```

```
Ran 21 tests for test/concrete/ConcreteDepositCap.t.sol:ConcreteAsyncVault_DepositCap_Test
[PASS] test_atTheCap_rebalanceIsIdempotent() (gas: 677287)
[PASS] test_atTheCap_yieldNeverGrowsThePosition() (gas: 1225792)
[PASS] test_capAboveAllocation_isInert() (gas: 295720)
[PASS] test_capHeadroomBelowMinDeposit_staysUnderTheCap() (gas: 1038755)
[PASS] test_capHolds_whenPoolKeepsPushingCredit() (gas: 1001474)
[PASS] test_capLoweredWhileAPoolRedeemIsInFlight_topsUpTheOpenRequest() (gas: 902615)
[PASS] test_cappedAllocation_deploysUpToTheCapAndParksTheRest() (gas: 652842)
[PASS] test_depositCap_defaultsToUncapped() (gas: 15970)
[PASS] test_inFlightUnwind_keepsTvlAndProfitStable() (gas: 481871)
[PASS] test_lossAtTheCap_reopensHeadroomAndRedeploysIdle() (gas: 766787)
[PASS] test_loweringTheCap_convergesWithAnOpenCreditLine() (gas: 1121895)
[PASS] test_loweringTheCap_doesNotDisturbProfitAccounting() (gas: 866362)
[PASS] test_loweringTheCap_unwindsTheVaultPosition() (gas: 769805)
[PASS] test_overCapRedeem_isNotQueuedTwice() (gas: 513552)
[PASS] test_overCapRedeem_isNotStackedOnExcessDebt() (gas: 838991)
[PASS] test_queueSwitchedOffWithARequestStranded_capsTotalVaultExposure() (gas: 601201)
[PASS] test_setDepositCap_revertsForNonGovernor() (gas: 21104)
[PASS] test_setDepositCap_updatesAndEmits() (gas: 59659)
[PASS] test_uncapped_afterResettingCapToZero() (gas: 652895)
[PASS] test_uncapped_deploysEverythingAndNeverRepays() (gas: 258578)
[PASS] test_vaultCapTighterThanStrategyCap_wins() (gas: 662906)
Suite result: ok. 21 passed; 0 failed; 0 skipped; finished in 8.26ms (10.02ms CPU time)
```

```
Ran 59 tests for test/concrete/ConcreteAsyncVault.unit.t.sol:ConcreteAsyncVault_Unit_Test
[PASS] test_cancelRequest_bubblesTheVaultsRefusal() (gas: 443687)
[PASS] test_cancelRequest_keepsTrackingWhenAnEpochStillHoldsARemainder() (gas: 398191)
[PASS] test_cancelRequest_recoversARequestStrandedByTheQueueBeingSwitchedOff() (gas: 514287)
[PASS] test_cancelRequest_revertsForNonKeeper() (gas: 16321)
[PASS] test_cancelRequest_unwindsTheRequestAndDropsTracking() (gas: 417675)
[PASS] test_claim_dropsFullyCollectedEpoch() (gas: 477261)
[PASS] test_claim_oneBadEpochBlocksTheWholeBatch() (gas: 814364)
[PASS] test_claim_prunesProcessedEpochWithNothingClaimable() (gas: 445430)
[PASS] test_claim_survivesStaleAccounting() (gas: 483999)
```

```
[PASS] test_deposit_belowMinDepositAmountStaysIdleButCounted() (gas: 206566)
[PASS] test_fullExit_drainsPoolDebtAcrossEpochs() (gas: 578735)
[PASS] test_fullExit_strandsResidualBelowMinWithdrawAmount() (gas: 330949)
[PASS] test_instantLiquidity_dirtyPausedWordDoesNotRevert() (gas: 304768)
[PASS] test_instantLiquidity_dirtyVersionWordDegradesToNetting() (gas: 357526)
[PASS] test_instantLiquidity_excludesLockedSharesWhenQueueOff() (gas: 338519)
[PASS] test_instantLiquidity_excludesSyncFallbackWhileQueueIsLive() (gas: 317586)
[PASS] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 314994)
[PASS] test_instantLiquidity_legacyImpl_netsWhatTheVaultForgot() (gas: 321024)
[PASS] test_instantLiquidity_nettingFollowsTheVaultVersion() (gas: 325679)
[PASS] test_instantLiquidity_saturatesAtZeroAndStillRespectsTheFloor() (gas: 355203)
[PASS] test_instantLiquidity_unaffectedWhenVaultHasNoPauseSelector() (gas: 498276)
[PASS] test_instantLiquidity_zerosClaimableDuringPause() (gas: 474641)
[PASS] test_instantLiquidity_zerosSyncFallbackDuringPause() (gas: 317915)
[PASS] test_isVaultPaused_reflectsVaultState() (gas: 35561)
[PASS] test_mock_queuedWithdrawSettlesNothing() (gas: 241613)
[PASS] test_queueDisabledMidFlight_backlogStillCountsInTvl() (gas: 431211)
[PASS] test_queueOff_rebalanceAndWithdrawUseSyncPath() (gas: 378921)
[PASS] test_queueOff_withdrawRefusalIsSwallowed() (gas: 435267)
[PASS] test_queueReenabled_backlogSettlesNormally() (gas: 482446)
[PASS] test_rebalance_deferredProfitTripsMinProfit() (gas: 461869)
[PASS] test_rebalance_queueActiveNeverSyncWithdraws() (gas: 477921)
[PASS] test_rebalance_revertsAboveMaxLoss() (gas: 381624)
[PASS] test_rebalance_revertsBelowMinProfit() (gas: 455830)
[PASS] test_rebalance_skipsAutoRedeemWhenDepositLocked() (gas: 364907)
[PASS] test_rebalance_skipsClaimWhenVaultPaused() (gas: 635699)
[PASS] test_rebalance_skipsDepositWhenVaultRefuses() (gas: 197149)
[PASS] test_rebalance_skipsRedeemWhenVaultRefuses() (gas: 315251)
[PASS] test_rebalance_succeedsAfterDepositLockExpires() (gas: 484914)
[PASS] test_redeemRequest_clampStaysWithinMaxRedeemBelowPar() (gas: 512770)
[PASS] test_redeemRequest_clampedDrainStillConverges() (gas: 1031716)
[PASS] test_redeemRequest_clampsToMaxWithdrawAmount() (gas: 460859)
[PASS] test_redeemRequest_skippedBelowMinWithdrawAmount() (gas: 321325)
[PASS] test_requestRedeem_revertsWhenQueueInactive() (gas: 265562)
[PASS] test_sweep_revertsForReservedTokens() (gas: 41356)
[PASS] test_sweep_transfersUnrelatedToken() (gas: 535666)
[PASS] test_trackAndUntrackEpoch_revertForNonKeeper() (gas: 25290)
[PASS] test_trackEpoch_clearsRebalanceBlockedByMovedRequest() (gas: 529361)
[PASS] test_trackEpoch_recoversRequestMovedToLaterEpoch() (gas: 651726)
[PASS] test_trackEpoch_revertsForEpochWithoutRequest() (gas: 273395)
[PASS] test_trackEpoch_revertsWhenAlreadyTracked() (gas: 391862)
[PASS] test_tvl_gasStaysBoundedWithManyPendingEpochs() (gas: 3442035)
[PASS] test_untrackEpoch_dropsPermanentlyStuckEpoch() (gas: 683131)
[PASS] test_untrackEpoch_revertsWhenNotTracked() (gas: 18813)
[PASS] test_upgrade_revertsForNonGovernor() (gas: 4299161)
[PASS] test_upgrade_syncToAsync_preservesStorageAndEnablesQueue() (gas: 10837588)
[PASS] test_withdraw_isSizedToWhatTheVaultCanActuallyPay() (gas: 348881)
[PASS] test_withdraw_legacyImpl_isSizedToWhatTheVaultCanActuallyPay() (gas: 355258)
[PASS] test_withdraw_queueActiveNeverSyncWithdraws() (gas: 297429)
[PASS] test_withdraw_survivesStaleAccountingInBothQueueStates() (gas: 336882)
Suite result: ok. 59 passed; 0 failed; 0 skipped; finished in 22.41ms (19.60ms CPU time)
```

Ran 49 tests for

test/concrete/ConcreteAsyncVault.ethereum.t.sol:ConcreteAsyncVault_ctRWA_USD1_Ethereum_Test

```
[PASS] test_cancelRequest_onlyKeeper() (gas: 11439)
[PASS] test_cancelRequest_unwindsARequestFromTheOpenEpoch() (gas: 579759)
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 5199476)
[PASS] test_instantLiquidity_excludesUnprocessedQueue() (gas: 708902)
[SKIP] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 0)
[SKIP] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 0)
[PASS] test_isReservedToken() (gas: 23992)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[PASS] test_multiEpoch_partialSettlementClaimsOnlyProcessed() (gas: 971890)
[PASS] test_multiEpoch_twoUnprocessedEpochsAreNettedOut() (gas: 862394)
[PASS] test_rebalance_additionalDeposit() (gas: 491387)
[PASS] test_rebalance_autoQueuesRedeemForExcessDebt() (gas: 619292)
[PASS] test_rebalance_claimsProcessedEpochs() (gas: 819388)
[SKIP] test_rebalance_defaultCapBlocksRoundingDust() (gas: 0)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 416594)
[PASS] test_rebalance_doesNotDoubleQueue() (gas: 690533)
[PASS] test_rebalance_emptyQueueIsNoOp() (gas: 112813)
[PASS] test_rebalance_firstDeposit() (gas: 385934)
```

```
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 509292)
[PASS] test_rebalance_noChangeWhenStable() (gas: 419369)
[PASS] test_rebalance_queuesUnreportedProfit() (gas: 794593)
[PASS] test_rebalance_realizesProfitAfterSettlement() (gas: 1016586)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 467201)
[PASS] test_rebalance_reportsLossWithExcessDebt() (gas: 667472)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 815051)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 428404)
[PASS] test_rebalance_skipsDepositFailure() (gas: 315196)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 296959)
[SKIP] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 0)
[PASS] test_reportLoss_onlyGovernor() (gas: 14317)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 384754)
[PASS] test_reportLoss_writesDownTheDebt() (gas: 414079)
[PASS] test_requestRedeem_movesSharesOutOfBalance() (gas: 542689)
[PASS] test_requestRedeem_onlyKeeper() (gas: 11467)
[PASS] test_requestRedeem_revertsAboveShareBalance() (gas: 405436)
[SKIP] test_requestRedeem_revertsWhenQueueInactive() (gas: 0)
[PASS] test_requestRedeem_zeroAmount() (gas: 18122)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50953)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17093)
[PASS] test_settleThenRebalance_paysPoolBack() (gas: 776132)
[PASS] test_tvl_invariantAcrossQueueLifecycle() (gas: 823235)
[SKIP] test_vault_refusesDepositWhenCapIsClosed() (gas: 0)
[SKIP] test_withdraw_all_fromDeposit() (gas: 0)
[PASS] test_withdraw_fromBalance() (gas: 295368)
[PASS] test_withdraw_queueActive_doesNotQueue() (gas: 400529)
[SKIP] test_withdraw_skipsWithdrawFailure() (gas: 0)
[SKIP] test_withdraw_some_fromDeposit() (gas: 0)
[SKIP] test_withdraw_survivesDepositLock() (gas: 0)
[PASS] test_withdraw_survivesStaleAccounting() (gas: 459235)
Suite result: ok. 39 passed; 0 failed; 10 skipped; finished in 892.80ms (40.42ms CPU time)
```

Ran 29 tests for test/concrete/ConcreteVault.ethereum.t.sol:ConcreteVault_ctFrontierUSDC_Ethereum_Test

```
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 3846315)
[PASS] test_instantLiquidity_matchesWithdrawablePosition() (gas: 534086)
[PASS] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 448247)
[PASS] test_isReservedToken() (gas: 24057)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[PASS] test_rebalance_additionalDeposit() (gas: 623291)
[PASS] test_rebalance_defaultCapBlocksRoundingDust() (gas: 453244)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 501835)
[PASS] test_rebalance_firstDeposit() (gas: 452874)
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 571745)
[PASS] test_rebalance_noChangeWhenStable() (gas: 517742)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 541543)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 1015293)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 506440)
[PASS] test_rebalance_skipsDepositFailure() (gas: 306422)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 288193)
[SKIP] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 0)
[PASS] test_reportLoss_onlyGovernor() (gas: 14428)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 454420)
[PASS] test_reportLoss_writesDownTheDebt() (gas: 490501)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50930)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17159)
[SKIP] test_vault_refusesDepositWhenCapIsClosed() (gas: 0)
[PASS] test_withdraw_all_fromDeposit() (gas: 575281)
[PASS] test_withdraw_fromBalance() (gas: 288437)
[PASS] test_withdraw_skipsWithdrawFailure() (gas: 501569)
[PASS] test_withdraw_some_fromDeposit() (gas: 553691)
[PASS] test_withdraw_survivesDepositLock() (gas: 544415)
[PASS] test_withdraw_survivesStaleAccounting() (gas: 557855)
Suite result: ok. 27 passed; 0 failed; 2 skipped; finished in 3.16s (1.41s CPU time)
```

Ran 8 tests for

test/concrete/ConcreteAsyncManagedVault.ethereum.t.sol:ConcreteAsyncManagedVault_ctDefiUSDT_Ethereum_Test

```
[PASS] test_fork_depositToVault_reachesLiveVault() (gas: 383171)
[PASS] test_fork_firstRebalanceAfterFullDeposit_revertsWhenLossCapUnset() (gas: 343173)
[PASS] test_fork_firstRebalanceAfterFullDeposit_survivesWithLossCap() (gas: 388350)
[PASS] test_fork_instantLiquidity_isIdleOnly() (gas: 368264)
[PASS] test_fork_rebalance_movesNoPrincipal() (gas: 289067)
```



```
[PASS] test_fork_requestRedeem_opensLiveEpoch() (gas: 466693)
[PASS] test_fork_userWithdraw_neverExitsLiveVault() (gas: 400100)
[PASS] test_fork_withdrawFromVault_claimsAfterSettlement() (gas: 882378)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 5.00s (4.65s CPU time)
```

Ran 49 tests for

test/concrete/ConcreteAsyncVault.ethereum.t.sol:ConcreteAsyncVault_ctFrontierUSDC_Ethereum_Test

```
[PASS] test_cancelRequest_onlyKeeper() (gas: 11439)
[SKIP] test_cancelRequest_unwindsARequestFromTheOpenEpoch() (gas: 0)
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 5199476)
[SKIP] test_instantLiquidity_excludesUnprocessedQueue() (gas: 0)
[PASS] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 520554)
[PASS] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 457537)
[PASS] test_isReservedToken() (gas: 23992)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[SKIP] test_multiEpoch_partialSettlementClaimsOnlyProcessed() (gas: 0)
[SKIP] test_multiEpoch_twoUnprocessedEpochsAreNettedOut() (gas: 0)
[PASS] test_rebalance_additionalDeposit() (gas: 635154)
[SKIP] test_rebalance_autoQueuesRedeemForExcessDebt() (gas: 0)
[SKIP] test_rebalance_claimsProcessedEpochs() (gas: 0)
[PASS] test_rebalance_defaultCapBlocksRoundingDust() (gas: 462561)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 509443)
[SKIP] test_rebalance_doesNotDoubleQueue() (gas: 0)
[PASS] test_rebalance_emptyQueueIsNoOp() (gas: 112351)
[PASS] test_rebalance_firstDeposit() (gas: 460901)
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 583296)
[PASS] test_rebalance_noChangeWhenStable() (gas: 529605)
[SKIP] test_rebalance_queuesUnreportedProfit() (gas: 0)
[SKIP] test_rebalance_realizesProfitAfterSettlement() (gas: 0)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 549984)
[SKIP] test_rebalance_reportsLossWithExcessDebt() (gas: 0)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 1029946)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 513504)
[PASS] test_rebalance_skipsDepositFailure() (gas: 312516)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 294280)
[SKIP] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 0)
[PASS] test_reportLoss_onlyGovernor() (gas: 14317)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 460116)
[PASS] test_reportLoss_writesDownTheDebt() (gas: 496775)
[SKIP] test_requestRedeem_movesSharesOutOfBalance() (gas: 0)
[PASS] test_requestRedeem_onlyKeeper() (gas: 11467)
[PASS] test_requestRedeem_revertsAboveShareBalance() (gas: 468729)
[PASS] test_requestRedeem_revertsWhenQueueInactive() (gas: 480645)
[PASS] test_requestRedeem_zeroAmount() (gas: 18122)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50953)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17093)
[SKIP] test_settleThenRebalance_paysPoolBack() (gas: 0)
[SKIP] test_tv1_invariantAcrossQueueLifecycle() (gas: 0)
[SKIP] test_vault_refusesDepositWhenCapIsClosed() (gas: 0)
[PASS] test_withdraw_all_fromDeposit() (gas: 589700)
[PASS] test_withdraw_fromBalance() (gas: 292134)
[SKIP] test_withdraw_queueActive_doesNotQueue() (gas: 0)
[PASS] test_withdraw_skipsWithdrawFailure() (gas: 519044)
[PASS] test_withdraw_some_fromDeposit() (gas: 565974)
[PASS] test_withdraw_survivesDepositLock() (gas: 571589)
[PASS] test_withdraw_survivesStaleAccounting() (gas: 582961)
```

Suite result: ok. 33 passed; 0 failed; 16 skipped; finished in 5.95s (1.62s CPU time)

Ran 49 tests for

test/concrete/ConcreteAsyncVault.ethereum.t.sol:ConcreteAsyncVault_srRoyUSDC_Ethereum_Test

```
[PASS] test_cancelRequest_onlyKeeper() (gas: 11439)
[PASS] test_cancelRequest_unwindsARequestFromTheOpenEpoch() (gas: 572994)
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 5199356)
[PASS] test_instantLiquidity_excludesUnprocessedQueue() (gas: 634946)
[SKIP] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 0)
[SKIP] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 0)
[PASS] test_isReservedToken() (gas: 23992)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[PASS] test_multiEpoch_partialSettlementClaimsOnlyProcessed() (gas: 933346)
[PASS] test_multiEpoch_twoUnprocessedEpochsAreNettedOut() (gas: 823460)
[PASS] test_rebalance_additionalDeposit() (gas: 507243)
[PASS] test_rebalance_autoQueuesRedeemForExcessDebt() (gas: 636129)
```



```
[PASS] test_rebalance_claimsProcessedEpochs() (gas: 762312)
[PASS] test_rebalance_defaultCapBlocksRoundingDust() (gas: 379157)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 406230)
[PASS] test_rebalance_doesNotDoubleQueue() (gas: 712895)
[PASS] test_rebalance_emptyQueueIsNoOp() (gas: 118087)
[PASS] test_rebalance_firstDeposit() (gas: 377829)
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 498545)
[PASS] test_rebalance_noChangeWhenStable() (gas: 432792)
[PASS] test_rebalance_queuesUnreportedProfit() (gas: 785308)
[PASS] test_rebalance_realizesProfitAfterSettlement() (gas: 977123)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 456303)
[PASS] test_rebalance_reportsLossWithExcessDebt() (gas: 652166)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 806636)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 419824)
[PASS] test_rebalance_skipsDepositFailure() (gas: 314576)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 302127)
[PASS] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 315093)
[PASS] test_reportLoss_onlyGovernor() (gas: 14317)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 374426)
[PASS] test_reportLoss_writesDownTheDebt() (gas: 403751)
[PASS] test_requestRedeem_movesSharesOutOfBalance() (gas: 522171)
[PASS] test_requestRedeem_onlyKeeper() (gas: 11467)
[PASS] test_requestRedeem_revertsAboveShareBalance() (gas: 394225)
[SKIP] test_requestRedeem_revertsWhenQueueInactive() (gas: 0)
[PASS] test_requestRedeem_zeroAmount() (gas: 18122)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50953)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17093)
[PASS] test_settleThenRebalance_paysPoolBack() (gas: 745410)
[PASS] test_tvl_invariantAcrossQueueLifecycle() (gas: 789255)
[PASS] test_vault_refusesDepositWhenCapIsClosed() (gas: 307566)
[SKIP] test_withdraw_all_fromDeposit() (gas: 0)
[PASS] test_withdraw_fromBalance() (gas: 298305)
[PASS] test_withdraw_queueActive_doesNotQueue() (gas: 389814)
[SKIP] test_withdraw_skipsWithdrawFailure() (gas: 0)
[SKIP] test_withdraw_some_fromDeposit() (gas: 0)
[SKIP] test_withdraw_survivesDepositLock() (gas: 0)
[SKIP] test_withdraw_survivesStaleAccounting() (gas: 0)
Suite result: ok. 41 passed; 0 failed; 8 skipped; finished in 6.03s (9.11s CPU time)
```

Ran 49 tests for

test/concrete/ConcreteAsyncVault.ethereum.t.sol:ConcreteAsyncVault_ctDeltaWeETH_Ethereum_Test

```
[PASS] test_cancelRequest_onlyKeeper() (gas: 11439)
[PASS] test_cancelRequest_unwindsARequestFromTheOpenEpoch() (gas: 546416)
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 5199356)
[PASS] test_instantLiquidity_excludesUnprocessedQueue() (gas: 660861)
[SKIP] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 0)
[SKIP] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 0)
[PASS] test_isReservedToken() (gas: 23992)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[PASS] test_multiEpoch_partialSettlementClaimsOnlyProcessed() (gas: 902502)
[PASS] test_multiEpoch_twoUnprocessedEpochsAreNettedOut() (gas: 800712)
[PASS] test_rebalance_additionalDeposit() (gas: 450128)
[PASS] test_rebalance_autoQueuesRedeemForExcessDebt() (gas: 569552)
[PASS] test_rebalance_claimsProcessedEpochs() (gas: 769309)
[SKIP] test_rebalance_defaultCapBlocksRoundingDust() (gas: 0)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 380071)
[PASS] test_rebalance_doesNotDoubleQueue() (gas: 633883)
[PASS] test_rebalance_emptyQueueIsNoOp() (gas: 100364)
[PASS] test_rebalance_firstDeposit() (gas: 357775)
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 465253)
[PASS] test_rebalance_noChangeWhenStable() (gas: 390740)
[PASS] test_rebalance_queuesUnreportedProfit() (gas: 747296)
[PASS] test_rebalance_realizesProfitAfterSettlement() (gas: 966776)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 435359)
[PASS] test_rebalance_reportsLossWithExcessDebt() (gas: 615125)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 761360)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 399166)
[PASS] test_rebalance_skipsDepositFailure() (gas: 300920)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 291260)
[PASS] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 307284)
[PASS] test_reportLoss_onlyGovernor() (gas: 14317)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 360843)
```

```
[PASS] test_reportLoss_writesDownTheDebt() (gas: 390168)
[PASS] test_requestRedeem_movesSharesOutOfBalance() (gas: 501639)
[PASS] test_requestRedeem_onlyKeeper() (gas: 11467)
[PASS] test_requestRedeem_revertsAboveShareBalance() (gas: 377835)
[SKIP] test_requestRedeem_revertsWhenQueueInactive() (gas: 0)
[PASS] test_requestRedeem_zeroAmount() (gas: 18122)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50953)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17093)
[PASS] test_settleThenRebalance_paysPoolBack() (gas: 738717)
[PASS] test_tvl_invariantAcrossQueueLifecycle() (gas: 766024)
[PASS] test_vault_refusesDepositWhenCapIsClosed() (gas: 271916)
[SKIP] test_withdraw_all_fromDeposit() (gas: 0)
[PASS] test_withdraw_fromBalance() (gas: 289135)
[PASS] test_withdraw_queueActive_doesNotQueue() (gas: 377340)
[SKIP] test_withdraw_skipsWithdrawFailure() (gas: 0)
[SKIP] test_withdraw_some_fromDeposit() (gas: 0)
[SKIP] test_withdraw_survivesDepositLock() (gas: 0)
[PASS] test_withdraw_survivesStaleAccounting() (gas: 424966)
Suite result: ok. 41 passed; 0 failed; 8 skipped; finished in 6.03s (8.32s CPU time)
```

Ran 49 tests for

test/concrete/ConcreteAsyncVault.ethereum.t.sol:ConcreteAsyncVault_ctDefiUSDT_Ethereum_Test

```
[PASS] test_cancelRequest_onlyKeeper() (gas: 11439)
[PASS] test_cancelRequest_unwindsARequestFromTheOpenEpoch() (gas: 519828)
[PASS] test_initialize_revertsOnAssetMismatch() (gas: 5199476)
[PASS] test_instantLiquidity_excludesUnprocessedQueue() (gas: 886860)
[SKIP] test_instantLiquidity_followsWhatTheVaultAdvertises() (gas: 0)
[SKIP] test_instantLiquidity_zeroBelowMinWithdrawAmount() (gas: 0)
[PASS] test_isReservedToken() (gas: 23992)
[PASS] test_maxLossBps_defaultsToZero() (gas: 11189)
[PASS] test_multiEpoch_partialSettlementClaimsOnlyProcessed() (gas: 1153576)
[PASS] test_multiEpoch_twoUnprocessedEpochsAreNettedOut() (gas: 823061)
[PASS] test_rebalance_additionalDeposit() (gas: 471982)
[PASS] test_rebalance_autoQueuesRedeemForExcessDebt() (gas: 571927)
[PASS] test_rebalance_claimsProcessedEpochs() (gas: 1005064)
[PASS] test_rebalance_defaultCapBlocksRoundingDust() (gas: 355742)
[PASS] test_rebalance_depositsHeadroomWhenCapIsTight() (gas: 377411)
[PASS] test_rebalance_doesNotDoubleQueue() (gas: 642048)
[PASS] test_rebalance_emptyQueueIsNoOp() (gas: 107221)
[PASS] test_rebalance_firstDeposit() (gas: 354168)
[PASS] test_rebalance_lossAtExactCapIsAccepted() (gas: 467503)
[PASS] test_rebalance_noChangeWhenStable() (gas: 408884)
[PASS] test_rebalance_queuesUnreportedProfit() (gas: 717817)
[PASS] test_rebalance_realizesProfitAfterSettlement() (gas: 1120009)
[PASS] test_rebalance_reportsLossOnceCapIsOpened() (gas: 432523)
[PASS] test_rebalance_reportsLossWithExcessDebt() (gas: 592010)
[PASS] test_rebalance_reportsProfitAgainstCreditLine() (gas: 755886)
[PASS] test_rebalance_revertsWhenLossExceedsCap() (gas: 396291)
[PASS] test_rebalance_skipsDepositFailure() (gas: 311032)
[PASS] test_rebalance_skipsDepositWhenCapIsClosed() (gas: 292818)
[SKIP] test_rebalance_skipsDepositWhenCapIsClosedOnChain() (gas: 0)
[PASS] test_reportLoss_onlyGovernor() (gas: 14317)
[PASS] test_reportLoss_revertsAboveTotalDebt() (gas: 354304)
[PASS] test_reportLoss_writesDownTheDebt() (gas: 383628)
[PASS] test_requestRedeem_movesSharesOutOfBalance() (gas: 470412)
[PASS] test_requestRedeem_onlyKeeper() (gas: 11467)
[PASS] test_requestRedeem_revertsAboveShareBalance() (gas: 374975)
[SKIP] test_requestRedeem_revertsWhenQueueInactive() (gas: 0)
[PASS] test_requestRedeem_zeroAmount() (gas: 18122)
[PASS] test_setMaxLossBps_onlyGovernor() (gas: 50953)
[PASS] test_setMaxLossBps_rejectsAboveMaxBps() (gas: 17093)
[PASS] test_settleThenRebalance_paysPoolBack() (gas: 992637)
[PASS] test_tvl_invariantAcrossQueueLifecycle() (gas: 1027972)
[SKIP] test_vault_refusesDepositWhenCapIsClosed() (gas: 0)
[SKIP] test_withdraw_all_fromDeposit() (gas: 0)
[PASS] test_withdraw_fromBalance() (gas: 289884)
[PASS] test_withdraw_queueActive_doesNotQueue() (gas: 368558)
[SKIP] test_withdraw_skipsWithdrawFailure() (gas: 0)
[SKIP] test_withdraw_some_fromDeposit() (gas: 0)
[SKIP] test_withdraw_survivesDepositLock() (gas: 0)
```

Code Coverage

File	% Lines	% Statements	% Branches	% Funcs
contracts/strategies/concrete/ConcreteAsyncManagedVault.sol	100.00% (22/22)	100.00% (25/25)	100.00% (2/2)	100.00% (5/5)
contracts/strategies/concrete/ConcreteAsyncVault.sol	100.00% (152/152)	99.09% (218/220)	94.59% (35/37)	100.00% (22/22)
contracts/strategies/concrete/ConcreteVault.sol	92.98% (106/114)	93.33% (140/150)	85.00% (17/20)	90.91% (20/22)

Changelog

- 2026-08-31 - Initial report
- 2026-09-22 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp’s team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp’s collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the

extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

